World Models in Action

3. Representation learning before generation

This chapter covers

- Learning representations without labels
- Contrastive learning, derived and trained
- Collapse and its remedies
- Masked prediction and latent bottlenecks
- Measuring and assembling a state representation

A world model needs a representation of the current situation that preserves the distinctions needed to predict what happens next. An encoder must learn that representation from observations such as images, even when the training data do not label position, velocity, or other physical quantities.

Self-supervised learning turns structure in observations into training signals. A model can identify a transformation, match two views of a frame, reconstruct hidden pixels, or use a context to identify a future representation. These objectives ask for different things, so their success does not automatically imply that the resulting state is useful for prediction or planning.

We compare those learning signals using rendered tasks from the DeepMind Control Suite. The encoders train on images; simulator state variables are reserved for post-training probes, except for a clearly separated supervised reference. The goal is to measure what the learned representations retain, not to assume that any objective must succeed.

The runners select CUDA automatically when PyTorch detects a compatible GPU; otherwise, they use the CPU. Rendering is separate from model training. Environment setup, rendering dependencies, and experiment commands are documented in the code guide.

Reported uncertainty is the standard deviation across trained model seeds, not across episodes or trajectories. Each experiment's results file records the seeds and configuration used.

The chapter moves from pretext tasks to contrastive and non-contrastive objectives, masked prediction, latent-capacity experiments, and temporal prediction. It ends by connecting the learned encoder to the larger state–dynamics–planning system.[sidebar] .Three components, three different jobs

> A **static visual encoder** maps the current image, or a fixed stack of images, to a latent vector z_t. It has no memory across calls. A **recurrent wrapper** is optional: it takes the current visual latent and a previous hidden state h_{t-1}, then returns an updated hidden state h_t. That hidden state is a learned model state, not the simulator's physical state. A **full world model** additionally needs a transition model that predicts future state or observations conditioned on actions, and components that evaluate or plan over possible futures. The GRU wrapper and CPC experiment in this chapter do not, by themselves, implement that full system.

3.1. Learning a state without labels

Chapter 2 established the state contract: a representation should preserve decision-relevant distinctions while discarding irrelevant visual detail. This section asks how to learn such a representation from unlabeled observations.

Let x denote an observation, such as an image rendered by a simulator. An encoder f_θ, parameterized by θ, transforms it into a representation z:

$$\boxed{z = f_\theta(x)}$$

The encoder maps an observation to a vector of d numbers. Sufficiency means retaining information needed for downstream prediction; minimality means not carrying detail that adds no value for the task. A representation can fail in either direction: it may preserve too much raw appearance, or compress away a distinction needed to predict what happens next.

3.1.1. Aliasing: When compression loses the wrong information

Aliasing occurs when observations that require different predictions map to the same representation. For example, visually similar cart-pole frames may correspond to opposite cart

velocities. The encoder then discards a distinction that a downstream model cannot recover from the representation alone.

figure 3.1 is a map of the major self-supervised training signals, not an exhaustive taxonomy or a ranking of methods.

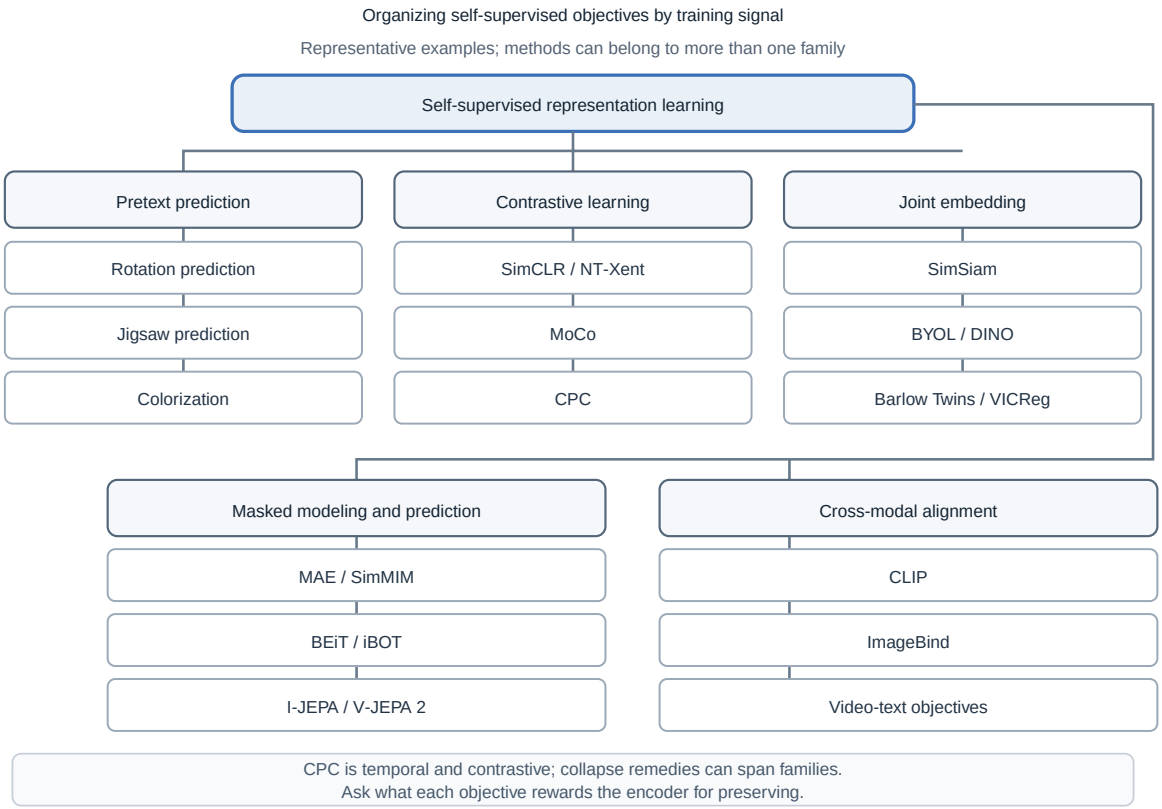


Figure 3.1 Self-supervised learning: a map of training signals

table 3.1 gathers the notation used in the objectives, probe definitions, and experiment descriptions.

Table 3.1 Notation used in this chapter

Symbol	**Meaning**	**Use in this chapter**
x	Rendered observation or frame	Static visual input
y	Simulator physics-state target	Evaluation only for self-supervised methods; training target for the separately labelled supervised reference

Symbol	Meaning	Use in this chapter
f_θ	Static visual encoder with parameters θ	Maps pixels to features
z	Latent representation	Output of the visual encoder
d	Latent dimension	Capacity sweep in §3.6
t	Time index	Temporal sequences
z_t	Visual latent at time t	CPC inputs and retrieval candidates
c_t	CPC context vector	Summary of the encoded past for future-latent scoring
h_t	Optional recurrent-wrapper hidden state	Interface state; not simulator physics state
N	Source-frame count in a contrastive batch	Two views produce $2N$ embeddings
K	Number of negatives per anchor	Standard NT-Xent convention: $K = 2N - 2$
τ	Contrastive temperature	Controls softmax sharpness
M	Mask indicator for a patch	MAE loss is applied on masked patches
W_k	Horizon-specific CPC compatibility matrix	Scores latent at horizon k against context
R^2	Held-out state-probe score	Linear decodability measure, not a complete measure of information

figure 3.2 illustrates how an encoder can preserve a distinction needed for prediction or collapse two situations into the same representation.

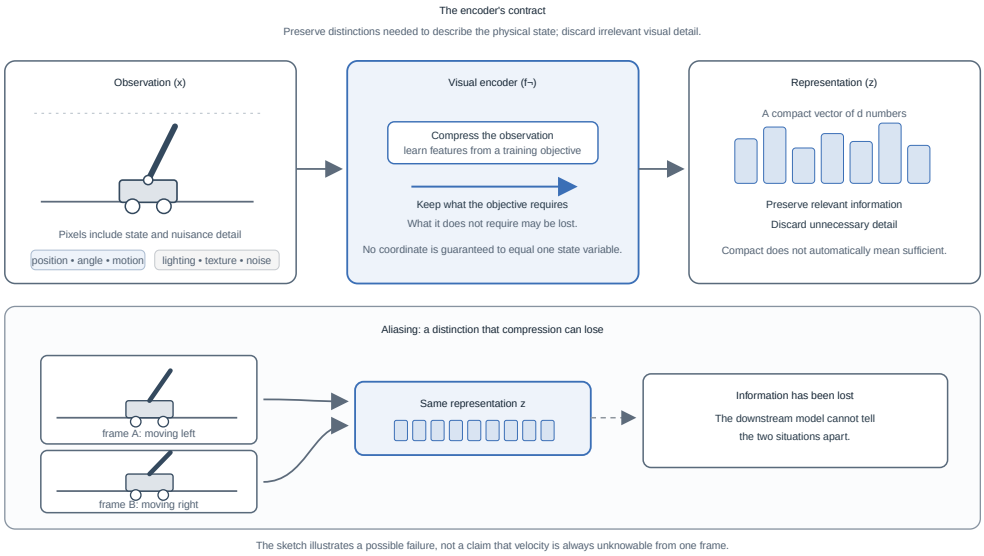


Figure 3.2 What a useful representation must preserve

The example illustrates a possibility, not a universal claim that velocity is impossible to infer from a single image. A single frame may contain visual or statistical clues correlated with velocity. Whether those clues are enough is a question to measure, not an assumption to make. Later in the chapter, we will compare state measurements from single-frame representations with those from stacked frames.

3.1.2. Why not train directly on the state?

When physical labels are available, an encoder can be trained to predict them directly. In many settings, however, exact state labels are costly or unavailable, and the chosen labels may omit information needed by another task.

Self-supervised learning constructs its target from the observations or a transformation of them. Rotation class, a second view, a hidden image patch, or a future embedding can provide a training signal without a person labelling every frame. The signal is automatic, but the target still reflects a design choice: solving it need not produce a state that is sufficient for control.

3.1.3. The objective determines what the encoder is rewarded for keeping

The objective determines which relationships the encoder must preserve to reduce its loss. Contrastive learning rewards information shared across chosen views; masked prediction rewards information useful for reconstructing hidden content; temporal prediction rewards information useful for distinguishing or anticipating later observations.

The principle behind representation learning

A representation-learning objective encourages the model to retain information that helps solve its training task. Information not required by that task may be discarded. Architecture, data, augmentations, and training budget also matter, so a falling loss is not proof that the learned state is useful for prediction or planning.

3.1.4. Why use a benchmark rather than a toy world?

The experiments in this chapter use the DeepMind Control Suite (DMC), a collection of physics-based control tasks built on MuJoCo. The observations are rendered by the simulator rather than drawn by a custom world designed solely to demonstrate a particular learning method. The suite gives us both rendered images for training and simulator variables against which to evaluate the resulting representations.

Our experiments use three tasks, shown in figure 3.3.

- **Cartpole balance.** A pole is hinged to a cart that moves along a track. Its evaluation state has four variables: two generalized positions and two generalized velocities.
- **Finger spin.** A planar, two-joint finger interacts with a free body that can rotate. Its evaluation state has six variables: three generalized positions and three generalized velocities.
- **Cheetah run.** A planar, legged body moves across a surface. Its evaluation state has eighteen variables: nine generalized positions and nine generalized velocities.

Cartpole balance is the main task because its small state makes results relatively easy to interpret. Finger spin provides a second task for the objective comparison in Section 3.5. Cheetah run is used in Section 3.6 to examine how the size of the representation affects a task with a higher-dimensional state.

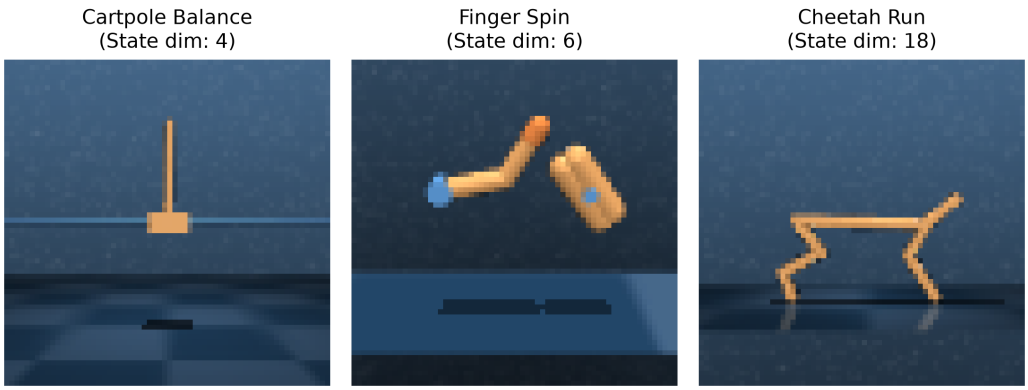


Figure 3.3 Rendered observations from the three tasks

3.1.5. Experimental setup and limits

The data-collection code records rendered frames and the corresponding simulator state. It samples actions uniformly within each task's action bounds, steps the environment, and caches the observations and state variables. The training and validation datasets are collected using independent environment seeds.

The key distinction is between training signal and evaluation target. For the self-supervised experiments, the training objectives receive rendered frames, not the stored physics-state labels. We use those labels to evaluate which physical quantities can be recovered from a learned representation. Section 3.5 also includes a separate supervised reference that is trained using state labels; that model is explicitly treated as a supervised comparison, not as a self-supervised method.

NOTE	The self-supervised objectives receive rendered images only; simulator state variables are held back for evaluation. The separately trained supervised reference uses state labels and is not a self-supervised method.

The common benchmark lets us compare the information made available by each objective without presuming that any objective succeeds.

3.2. Pretext tasks: Supervision from structure

A pretext task turns an unlabeled observation into a training example with a known target. The target comes from a transformation or relationship in the data, so we do not need a person to annotate each example.

Rotation prediction is a simple place to begin. It lets us construct the target explicitly, derive the loss from an ordinary classification problem, and then ask the central question of this chapter: does solving the pretext task require the encoder to preserve the information a world model needs?

3.2.1. Rotation prediction

Take a rendered cart-pole frame x. Rotate it by a multiple of 90 degrees and ask a network to predict which rotation was applied. We use four possible rotations: zero, 90, 180, and 270 degrees.

Let $r \in \{0, 1, 2, 3\}$ be the rotation label, where r counts quarter turns, and let $\rho_r(x)$ denote the corresponding rotated image. Starting with one frame gives us four training examples:

$$(\rho_0(x), 0), \quad (\rho_1(x), 1), \quad (\rho_2(x), 2), \quad (\rho_3(x), 3).$$

The label is known because we chose the transformation. No human annotation is needed.

Our frames are square, so rotating by a quarter turn simply rearranges the pixels. We do not need to introduce padding or interpolate new pixel values. The target is therefore determined by the

applied rotation, rather than by an artifact created at the image border.

Now we need a network that predicts the rotation. We already have an encoder f_θ from equation 3.1. Attach a classification head g_ϕ, with parameters ϕ, to map the representation to four scores:

$$a(x') = g_\phi(f_\theta(x')).$$

Here x' is a rotated input, and $a_j(x')$ is the score, or logit, assigned to rotation class j. A logit is not yet a probability. We convert the four scores into probabilities using softmax:

$$\frac{\exp(a_j(x'))}{\sum_{m=0}^{3} \exp(a_m(x'))}.$$

The four probabilities are nonnegative and sum to one. The model's task is to assign a high probability to the rotation that was actually applied.

How should we measure whether it does so? Suppose the correct label is r. Its predicted probability is $p_{\theta,\phi}(r \mid x')$. We penalize the network using the negative logarithm of that probability:

$$-\log p_{\theta,\phi}(r \mid x').$$

This is the familiar cross-entropy loss for a classification problem with one correct class. If the model assigns probability one to the correct answer, the loss is zero. If it assigns a very small probability to the correct answer, the loss is large.

We average this loss over the four rotations of each original frame, then over the training frames. The result is the rotation-prediction objective:

$$-\mathbb{E}_x\left[\frac{1}{4}\sum_{r=0}^{3} \log p_{\theta,\phi}(r \mid \rho_r(x))\right]$$

The expectation averages over the training observations. The parameters θ and ϕ are learned together: the encoder must produce features that help the classification head identify the rotation.

A useful reference point is uniform prediction. If the model assigns probability $1/4$ to every rotation, the loss for each example is

$$-\log\frac{1}{4} = \log 4 \approx 1.386.$$

The value is measured in nats, because we use the natural logarithm. A model that predicts the correct class confidently can achieve a lower loss.

Notice what we have done. We have not supplied the physics state y, the cart position, or the pole

angle as a training label. The objective only asks the model to identify the applied rotation. Equation 3.2 is an ordinary four-class classification loss; the special feature is how its labels are generated.

3.2.2. Two other pretext tasks

Rotation prediction is one example of using structure in the input to create a target. Two other classic tasks take a different approach.

Jigsaw prediction divides an image into a grid of tiles, shuffles them using a permutation selected from a predefined set, and trains the model to identify that permutation. The spatial relationships between tiles supply the learning signal. This can encourage features that capture local appearance and spatial arrangement, but it does not directly require the model to preserve physical variables such as velocity.

Colorization takes a grayscale image and trains a model to predict its color channels. The target is derived from the original image, so human labeling is unnecessary. Predicting color can encourage the model to use contextual information, but good color reconstruction is not the same as learning a state suitable for control. The objective rewards information useful for recovering color; it does not require every geometric or dynamical distinction a downstream world model needs.

figure 3.4 puts the three tasks side by side using the same cart-pole frame. Each task creates its own target, but the targets ask for different things.

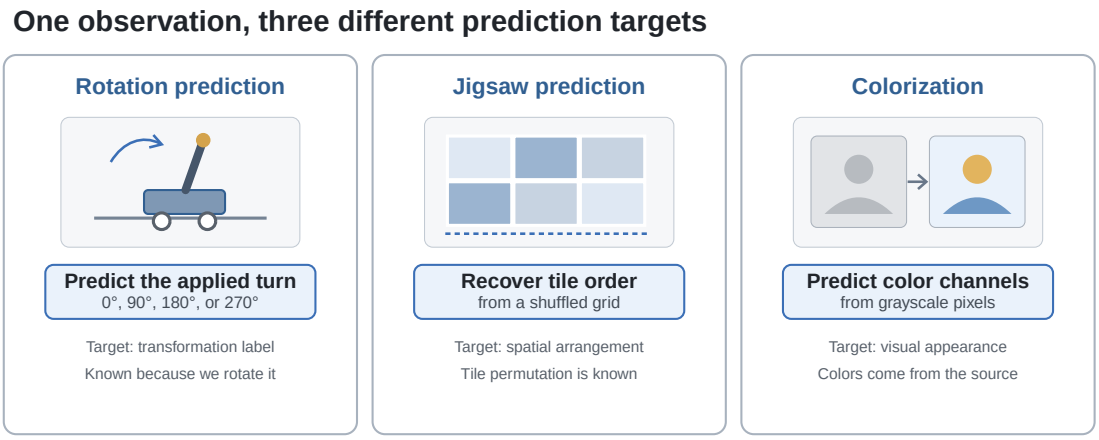


Figure 3.4 Three ways to create a learning target from one observation

Conceptual task categories informed by Gui et al. (2024). The diagram uses original schematics rather than an adapted published figure.

Gui and colleagues survey these and many other pretext tasks. This chapter uses rotation prediction because its target and loss are easy to derive. The purpose is not to cover every pretext

task, but to understand what a chosen target requires the encoder to learn.

3.2.3. Why a correct pretext answer may still be the wrong goal

Return to the principle from Section 3.1: the training objective determines what information the representation is encouraged to retain. Rotation prediction rewards features that help identify the rotation label. If the scene layout or background already makes that label easy to infer, the objective may be solvable without preserving the pole angle with the precision that control requires.

The same distinction applies to the other tasks. Jigsaw prediction rewards information useful for identifying tile arrangement. Colorization rewards information useful for predicting colors. Each objective can encourage useful features, and a trained encoder may retain more information than strictly necessary to solve its pretext task. But none of these losses, by itself, guarantees that the representation is sufficient for predicting the physical future.

That is the limitation we need to solve. Rather than ask the encoder to predict one fixed transformation, we want a learning signal that encourages it to ignore irrelevant changes while preserving the distinctions that matter. Small shifts or brightness changes might be nuisance variation; different physical situations should not be made indistinguishable merely because their images look similar.

This leads to instance discrimination, the starting point for contrastive learning. We will make two altered views of an observation and ask the model to recognize them as views of the same source. The choice of alterations will matter: it defines which changes the objective encourages the encoder to ignore. We can now derive the loss from first principles.

3.3. Contrastive learning from first principles

Contrastive learning asks whether two altered observations came from the same source. It constructs positive pairs from independent views of one frame and negatives from other frames, then trains an encoder to rank the positive above the alternatives.

The loss makes this identification task explicit. Its usefulness depends on the view transformations and candidate set: changes treated as irrelevant should preserve information the agent needs, and negatives may depict similar physical states.

3.3.1. Views, positives, and negatives

A view of an observation x is an altered copy of it. Write the view as

$$\tilde{x} = t(x),$$

where t is a randomly selected transformation. The transformation is applied to the image, not to

the underlying physical system.

In this chapter, the view pipeline uses two kinds of alteration. It shifts the image by a random integer offset of up to three pixels in each direction, then applies random brightness and contrast changes. The brightness offset is sampled from $[-0.1, 0.1]$, and the contrast factor from $[0.9, 1.1]$. The values are clipped to the valid image range.

These changes are deliberately modest. They alter pixel values and image alignment, but they are intended to leave the physical situation unchanged.

A positive pair consists of two independently transformed views of the same frame. A negative pair consists of views from different source frames. The distinction comes from how we created the data, so no human label is needed.

The training process operates on a batch of N source frames. Each source frame produces two views, giving $2N$ images. The encoder maps these images to representations $z_1, \ldots, z_{2N}$.

For a particular view i, call it the anchor. Its positive, written i^+, is the other view of the same original frame. The remaining views are its negatives. Excluding the anchor itself, there are $2N-1$ candidates: one positive and $2N-2$ negatives.

Thus,

$$K = 2N - 2,$$

where K is the number of negatives for each anchor. For example, a batch of $N = 3$ frames produces six views. Each anchor has one positive and four negatives.

figure 3.5 shows the positive view paired with an anchor and the negative candidates used in the batch.

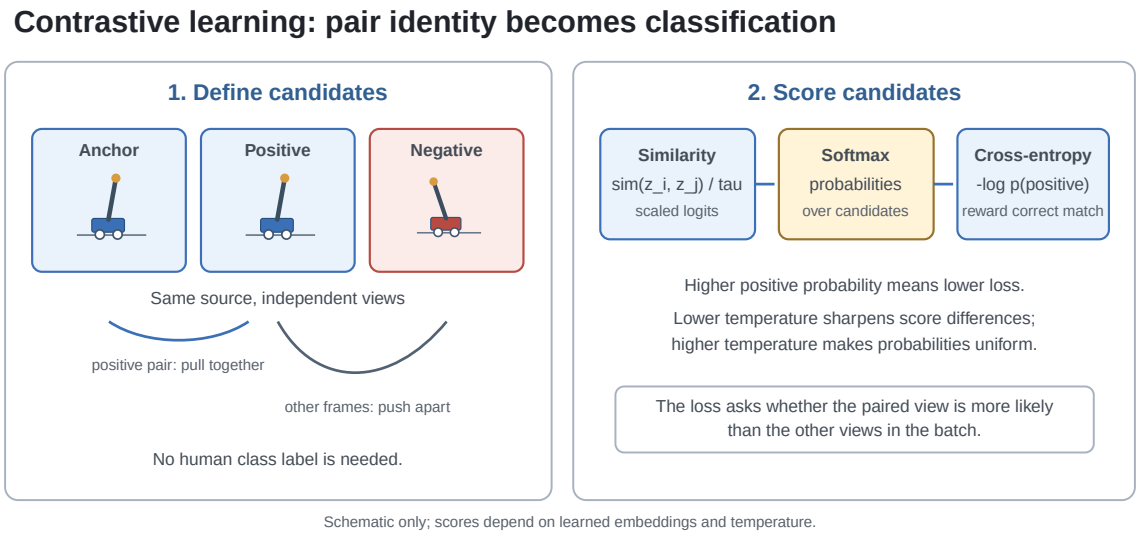


Figure 3.5 From paired views to a contrastive loss

The transformation t is a design decision, not a cosmetic detail. If the encoder must identify two views of the same frame, changes introduced randomly by t cannot reliably identify the source. The encoder is therefore encouraged to become insensitive to those changes.

This is the intended benefit of the augmentation pipeline. If brightness changes do not alter the physical state, the representation should ideally remain useful despite them. But the same reasoning also creates a risk: if the transformation changes something important, the objective may reward the encoder for ignoring it.

A shift of a few pixels may be harmless for a cart-pole. A shift large enough to move the cart or pole out of view could erase information needed to estimate the state. The positive-pair definition therefore specifies which changes should be treated as irrelevant. The following equations show how the loss enforces this requirement.

The excerpt is from worldmodels/data/augment.py. It uses the existing ViewPipeline implementation.

listing 3.1 shows how the batch pipeline constructs two views from each source observation.

Listing 3.1 Generating two views from a batch

```
def augment_single(self, images):
    x = random_crop_shift(
        images, max_shift=self.max_shift
    )                                                #A
    x = random_intensity_jitter(
        x,
        brightness_range=self.brightness_range,
        contrast_range=self.contrast_range,
    )                                                #B
    return x

def __call__(self, images):
    view_1 = self.augment_single(images)             #C
```

```
    view_2 = self.augment_single(images)
    return view_1, view_2
```

A Shifts each image by a randomly selected offset.

B Applies brightness and contrast jitter.

C Calls the pipeline twice, producing independent views.

The view pipeline is small, but its consequences run through the entire method. Contrastive learning can encourage invariance only to transformations that the training process actually applies. The choice of transformations is therefore part of the learning objective, even though it does not appear as a term in the loss equation.

NOTE

A batch drawn from trajectories can contain frames that are almost identical, such as neighbouring steps from one episode. The basic objective still treats views from different source frames as negatives. These are called false negatives: two observations may differ in source identity while sharing much of their physical state. These numbers describe a hypothetical uniform draw from the full 30,000-frame cache, assuming 1,000 frames per episode: the probability that another frame comes from the same episode is $999/29,999$, about 3.3%, and the chance of selecting a frame within five steps is at most $10/29,999$, about 0.033%. The primary contrastive run actually samples a 5,000-frame episode-stratified subset, so these percentages are illustrative for the full cache; the experiment does not report an observed false-negative frequency for its sampled pool. We keep the simple in-batch rule and report its results rather than silently changing the negative set.

3.3.2. From similarity to a loss

We now need a numerical measure of how well two representations match. We will build the objective in three steps: define a similarity score, turn the candidate views into a classification problem, and minimize the classification error.

Step 1: Measure similarity

We use cosine similarity. For two nonzero vectors u and v, it is the cosine of the angle between them:

$$(u, v) = \frac{u^\top v}{\| u \|_2 \, \| v \|_2}.$$

Cosine similarity lies between -1 and 1. A value near 1 means the vectors point in similar

directions; a value near -1 means they point in opposite directions. A value near zero means they are approximately orthogonal.

The normalization in the denominator means that the score depends on direction rather than vector length. Simply multiplying a representation by a positive constant cannot increase its cosine similarity with another vector.

Step 2: Turn similarity into a classification problem

Fix an anchor i. There are $2N-1$ candidate views, excluding the anchor itself. We want a classifier to assign the highest probability to the positive i^+.

For each candidate j, divide the similarity by a positive temperature τ, then apply softmax across the candidates:

$$p_i(j) = \frac{\exp\big((z_i, z_j)/\tau\big)}{\sum_{k \neq i} \exp\Big((z_i, z_k)/\tau\Big)}, \qquad j \neq i.$$

Here, $p_i(j)$ is the probability assigned to candidate j being the other view of the anchor's source frame. The correct answer is i^+.

The temperature τ controls how sharply differences in similarity affect the probabilities. A smaller temperature magnifies differences between candidates. A larger temperature makes their probabilities more uniform.

Step 3: Penalize a wrong answer

We use the negative logarithm of the probability assigned to the positive. The loss for anchor i is

$$\ell_i = -\log p_i(i^+).$$

A correct answer assigned high probability receives a small loss. A positive assigned low probability receives a large loss. Averaging across all $2N$ anchors gives the normalized temperature-scaled cross-entropy loss, or NT-Xent:

$$\frac{1}{2N}\sum_{i=1}^{2N} -\log p_i(i^+)$$

This is the same basic cross-entropy as in equation 3.2, but the classification problem is different. Rotation prediction chooses among four transformations. NT-Xent chooses among the other views in the batch, with a different correct class for each anchor.

Consider a batch with $N = 2$, so there are four views. Each anchor has three candidate views.

Suppose an anchor has cosine similarities of 0.80 with its positive and 0.10 and -0.30 with the two negatives. Let the temperature be $\tau = 0.5$.

Dividing the similarities by the temperature gives logits of 1.6, 0.2, and -0.6. Their exponentials are approximately 4.953, 1.221, and 0.549. The sum is 6.723, so the softmax assigns probabilities of approximately 0.737, 0.182, and 0.082.

The positive has probability 0.737, giving the anchor a loss of

$$-\log(0.737) \approx 0.306.$$

If the classifier cannot distinguish the positive and assigns the same probability to all three candidates, it assigns probability $1/3$ to the positive and has loss

$$-\log(1/3) = \log 3 \approx 1.099.$$

For general batch size N, an exactly uniform probability distribution over the candidates gives loss $\log(2N-1)$. This is the uninformative reference value for this classification problem.

WARNING

Compare NT-Xent loss values only when the batch size and temperature are the same. The number of candidate views changes with batch size, and the temperature changes the softmax probabilities. A lower numerical loss under a different configuration does not necessarily mean a better representation.

TIP

Batch size sets the number of negatives: $K = 2N - 2$. A larger batch asks the model to distinguish the positive from more candidates, but it also requires memory for more views and their similarity scores.

The combined schematic in figure 3.5 links the positive-pair construction to the candidate-classification form of NT-Xent. The derivation supplies the equations that the visual leaves implicit.

3.3.3. What InfoNCE actually optimizes

NT-Xent is one instance of the broader InfoNCE objective, used in contrastive predictive coding (van den Oord et al., 2018) and closely related to the contrastive formulation used by SimCLR (Chen et al., 2020). For each anchor, InfoNCE compares one positive with K negatives. Its mathematical properties help explain both what contrastive learning encourages and what it cannot guarantee.

First, we need a measure of shared information.

Mutual information

The mutual information $I(a; b)$ between random variables a and b measures how much knowing one reduces uncertainty about the other. It is the expected log ratio between their joint distribution and the product of their marginal distributions:

$$I(a; b) = \mathbb{E}_{p(a,\ b)}\left[\log \frac{p(a,\ b)}{p(a)p(b)}\right].$$

Mutual information is zero when a and b are independent. With natural logarithms, it is measured in nats.

Let a and b be the representations of two views of the same frame. This is the positive pair. Let $b_1, \ldots, b_K$ be representations of negative views drawn independently from the marginal distribution of b. For a positive score function (f$a,\ b$), InfoNCE is

$$-\mathbb{E}\left[\log \frac{f(a,\ b)}{f(a,\ b) + \sum_{k\ =\ 1}^{K} f(a,\ b_k)}\right].$$

For NT-Xent, the score uses exponentiated cosine similarity divided by temperature. Under the standard sampling assumptions, InfoNCE satisfies the lower bound

$$\boxed{I(a; b) \geq \log(K + 1) - \mathcal{L}_{\text{InfoNCE}}}$$

This is a result about the information shared by the two views' representations, not a direct guarantee that the representation captures the physical state.

Proof note. The mutual-information lower bound follows from the Bayes-optimal classifier over one positive and K marginal negatives. Appendix A gives the complete change-of-measure argument; here we use the bound only as an interpretation of shared view information, not as a guarantee that physical state is retained.

Equation 3.6 has two useful implications. First, lowering the InfoNCE loss raises the conventional lower bound on the information shared by the positive pair. Second, the bound is capped by $\log(K + 1)$, because the loss cannot be negative. A finite batch therefore limits how much shared information this objective can certify.

The assumptions matter. In the chapter's implementation, the negatives come from other frames in a batch, and two views from the same source frame are themselves related. Frames drawn from trajectories may also be correlated. The textbook mutual-information bound is therefore a useful interpretation of the objective, not an exact measurement of the trained model's mutual information.

The deeper limitation follows from how the views are constructed. Each view is a randomly transformed copy of the same source frame x. If z_1 and z_2 are their representations, the data-processing inequality gives

$$I(z_1; z_2) \leq I(\tilde{x}_1; \tilde{x}_2) \leq I(x; \tilde{x}_1).$$

The information shared by both views is constrained by what survives the transformations. A detail that changes independently between views cannot reliably identify a positive pair. Conversely, information preserved across both views can help.

But shared information is not the same as the information needed by a world model. The objective asks the encoder to distinguish one frame's views from other frames' views. It does not explicitly ask it to preserve every state variable useful for prediction or planning.

The gradient in equation 3.7 separates attraction toward the positive from repulsion by negatives. The coefficient of each negative is its current softmax probability, so hard negatives receive more weight. Appendix A derives the gradient and the cosine-similarity derivative step by step.

$$\nabla_{z_i} \ell_i = -\frac{1 - p_i(i^+)}{\tau} \nabla_{z_i}(z_i, z_{i^+}) + \sum_{k \notin i,\, i^+} \frac{p_i(k)}{\tau} \nabla_{z_i}(z_i, z_k).$$

The first term attracts the anchor toward its positive. The second term pushes it away from the negatives, weighted by their current probabilities.

Four observations follow.

Balanced forces: the sum of the negative probabilities is $1 - p_i(i^+)$, so the total negative weight equals the attraction weight for this anchor.

Self-limiting updates: when the positive already receives probability close to one, both attraction and repulsion become small.

Hard negatives matter most: a negative with high similarity receives a larger probability and a stronger push.

Temperature changes the emphasis: lowering τ makes the probability distribution more concentrated, increasing the relative influence of the most competitive negative.

In the numerical example above, the attraction weight is $(1 - 0.737) / 0.5 \approx 0.527$. The two negative weights are $0.182 / 0.5 = 0.364$ and $0.082 / 0.5 = 0.164$. Their sum is approximately equal to the attraction weight, with the small discrepancy due to rounding. Gradient descent applies the update

in the opposite direction of the gradient, increasing similarity to the positive and decreasing similarity to negatives.

equation 3.7 describes the anchor's role in one loss. The same representation also serves as a candidate in the losses of other anchors; those additional terms contribute to its complete gradient.

The key limitation remains: these gradients arrange the representations to solve the specified identification problem. The objective does not know whether the features used for identification are cart position, pole angle, texture, or some combination. That is why the following experiments evaluate the encoder rather than treating a falling loss as proof of a useful state.

3.3.4. Training a contrastive encoder

We now train the loss on rendered frames from cartpole_balance. The model has two parts: a convolutional encoder that produces the representation used later, and a projection head used during contrastive training.

The encoder in worldmodels/models/encoders.py contains four convolutional layers with 4×4 kernels and stride two. Starting with 64 × 64 RGB input, the spatial resolution is reduced as follows:

$$64 \to 32 \to 16 \to 8 \to 4.$$

The channel count grows from 32 to 64, 128, and 256. The final 256 × 4 × 4 feature map is flattened and projected to a 16-dimensional vector $z = f_\theta(x)$.

listing 3.2 isolates the encoder representation from the projection used by the contrastive loss.

Listing 3.2 The encoder representation and projection head

```
z = encoder(images)
p = projection_head(z)
```

The encoder maps each 64 × 64 RGB frame to a 16-dimensional representation. The projection head maps that representation to the space used by the contrastive loss. The distinction matters: downstream tasks use the encoder representation, not the projection used to train it.

The loss is applied to the projection $p = g_\phi(z)$, not directly to z. This follows the design used in SimCLR, where a nonlinear projection head is used during contrastive pretraining while the representation before that head is retained for downstream tasks (Chen et al., 2020). The reason for separating the two is practical: the projection can become invariant to some details needed by later tasks, while the encoder representation remains available to downstream components. This chapter adopts that setup without claiming that the two alternatives have been compared experimentally here.

listing 3.3 implements the vectorized NT-Xent objective derived above.

Listing 3.3 The NT-Xent loss

```
def nt_xent_loss(z1, z2, temperature=0.5):
    n = z1.shape[0]
    device = z1.device
    z1 = F.normalize(z1, dim=1)                          #A
    z2 = F.normalize(z2, dim=1)
    reps = torch.cat([z1, z2], dim=0)                    #B
    logits = torch.matmul(reps, reps.T) / temperature #C
    eye = torch.eye(2 * n, dtype=torch.bool, device=device)
    logits = logits.masked_fill(eye, -1e9)               #D
    labels = torch.cat([
        torch.arange(n, 2 * n, device=device),           #E
        torch.arange(0, n, device=device),
    ])
    return F.cross_entropy(logits, labels)               #F
```

A Normalizes each representation so its dot product becomes cosine similarity.

B Concatenates the two views into one batch of $2N$ vectors.

C Computes all pairwise similarities, scaled by the temperature.

D Removes self-similarity from each anchor's candidate set.

E Identifies the positive row for each view.

F Applies cross-entropy to the positive labels, implementing equation 3.5.

Masking the diagonal excludes the anchor itself from the softmax denominator. The label indices identify the other view of the same frame: the positive of row i is row $i+N$, and the positive of row $i+N$ is row i.

The repository also includes ch03/03_ntxent_from_scratch.py. It compares a direct loop-based implementation of the mathematical definition with the vectorized function above, checking both loss values and gradients. It is a verification script, rather than a second implementation intended for the main training run.

The training loop. The training script ch03/04_train_contrastive.py loads cached cart-pole frames, creates the two views, runs both through the shared encoder, applies the projection head, and evaluates NT-Xent. The optimizer updates the encoder and projection-head parameters together.

listing 3.4 shows the operations in one contrastive training update.

Listing 3.4 One contrastive update

```
view_a, view_b = view_pipeline(images)
z_a = encoder(view_a)
z_b = encoder(view_b)
p_a, p_b = projector(z_a), projector(z_b)
loss = nt_xent_loss(p_a, p_b)
optimizer.zero_grad()
loss.backward()
```

```
optimizer.step()
```

The two transformed views share the same encoder and projection head. The loss updates both networks; the physics-state labels are not part of this step.

The recorded run uses 5,000 training frames sampled evenly across the 30 training episodes, a batch size of 64, a 16-dimensional latent, Adam with learning rate 10^{-3}, temperature $\tau = 0.5$, and 15 epochs. The held-out validation set is sampled across the five validation episodes. The training frames are selected across episodes and across time within each episode; the script does not simply take the first 5,000 frames.

A batch contains $N = 64$ source frames, or 128 views, giving $K = 126$ negatives per anchor. Results are recorded for three seeds: 0, 1, and 2. The physics state is stored with each frame, but the contrastive training objective operates on the images and projected representations, not on that state.

figure 3.6 shows the recorded three-seed NT-Xent training loss on Cartpole. The mean falls from approximately 3.779 in the first epoch to 3.106 in the final epoch, with a seed standard deviation of about 0.005 at epoch 15. The curve is still decreasing at the final epoch, so this is the end of the chosen training budget, not a demonstrated convergence point.

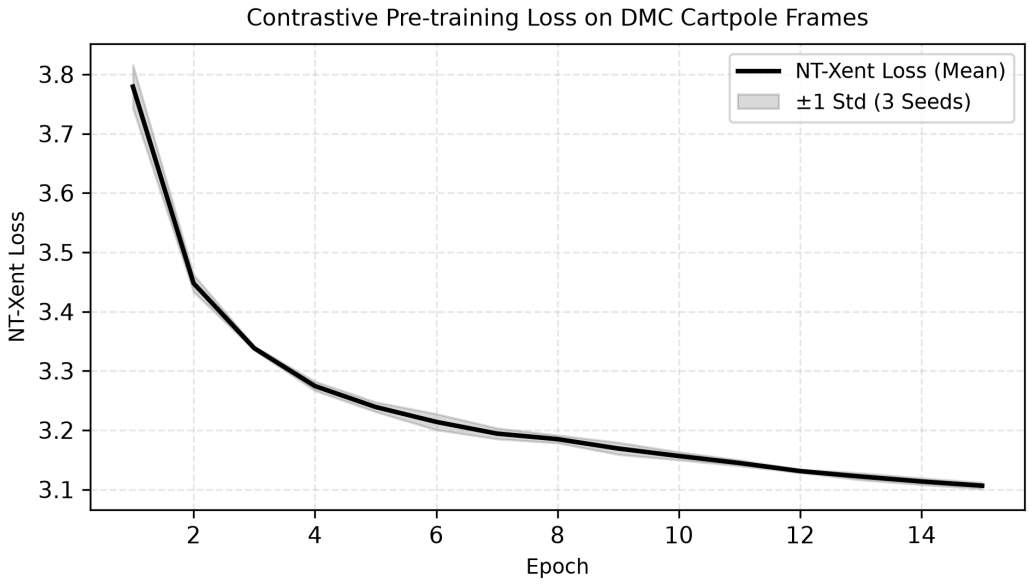


Figure 3.6 NT-Xent training loss on Cartpole

The loss is still decreasing at the final epoch, so this run ends at its chosen training budget rather than at a demonstrated convergence point. Its trend is evidence that the network is learning to solve the batch-identification task. It is not, by itself, evidence that the representation has captured the physical state.

A two-dimensional PCA projection can show broad geometric changes in embeddings, but it cannot establish which physical variables are recoverable. We therefore use held-out linear probes as the quantitative test of what the encoder retains.

3.3.5. Reading the state back with a linear probe

The simulator provides the physical state associated with each rendered frame. That state never enters the self-supervised loss. We can use it after training to ask which physical quantities are available in the representation.

A linear probe is a simple evaluation model fitted to a frozen encoder. We compute the representation z, freeze the encoder, and fit a linear map from z to the evaluation targets. We then evaluate that map on held-out observations. This measures the information that a linear readout can recover; it does not test whether an arbitrary nonlinear decoder could recover more.

For cart-pole balance, the raw state contains four variables: cart position, pole angle, cart velocity, and pole angular velocity. The pole angle needs special treatment. The simulator stores its hinge angle as an unwrapped value, so angles separated by a full revolution can have different numbers but represent the same physical orientation. An angle of 0.1 radians and one of $0.1 + 2\pi$ radians produce the same orientation.

To make a linear probe appropriate for this variable, the evaluation targets replace the angle θ with $\sin\theta$ and $\cos\theta$. The transformed target therefore has five components: cart position, $\sin\theta$, $\cos\theta$, cart velocity, and pole angular velocity. The probe reconstructs an angle using the inverse tangent, and the repository scores angle predictions with a circular R^2-like metric. This avoids treating a prediction across the angular wrap boundary as a large error simply because the numerical angle differs by nearly 2π.

Let $\check{z}_n$ be a representation standardized using the training-set feature means and standard deviations, and let $\tilde{y}_n$ be the transformed target vector. The probe fits a ridge-regression readout:

$$\min_{W,\, b} \sum_{n=1}^{M} \| \tilde{y}_n - W\check{z}_n - b \|_2^2 + \alpha \| W \|_F^2$$

Here, M is the number of training examples used to fit the probe; W and b are the linear weights and bias; and α controls the ridge penalty. The current probe implementation uses $\alpha = 10^{-3}$, standardizes the frozen features, and solves the ridge problem by least squares. The encoder is not updated when the probe is fitted.

listing 3.5 sketches the frozen-feature readout used for evaluation.

Listing 3.5 Evaluating a frozen representation

```
probe.fit(z_train, y_train)
y_pred = probe.predict(z_validation)
r2 = r2_score(y_validation, y_pred)
```

This sketch shows the evaluation logic, not the probe implementation. The repository standardizes features using training-set statistics, fits a ridge readout, and applies periodic scoring to angular targets where needed.

The score is computed on held-out validation frames. For ordinary state variables, $R^2 = 1$ is perfect prediction, $R^2 = 0$ matches the constant-mean baseline, and a negative value is worse than that baseline. Angular targets use the circular score described above.

A probe's limitations are important. A high score means the physical variable is available to a linear readout. A low or negative score does not prove the information is absent from z; it may be represented nonlinearly, or the probe may be limited by the data and fitting setup. The measurement tells us what is explicitly and linearly available under this evaluation, not everything the encoder may know.

Does adding a second frame recover velocity? The aliasing diagnostic compares a contrastive encoder receiving a single frame with one receiving two consecutive frames stacked along the channel dimension. Both are evaluated using a frozen-feature linear probe. The experiment uses three seeds, a latent dimension of 16, 12 training epochs, up to 2,000 training examples, and up to 500 validation examples. For the two-frame case, selected frame windows do not cross episode boundaries.

figure 3.7 compares the measured single-frame and two-frame state probes, including their variation across seeds.

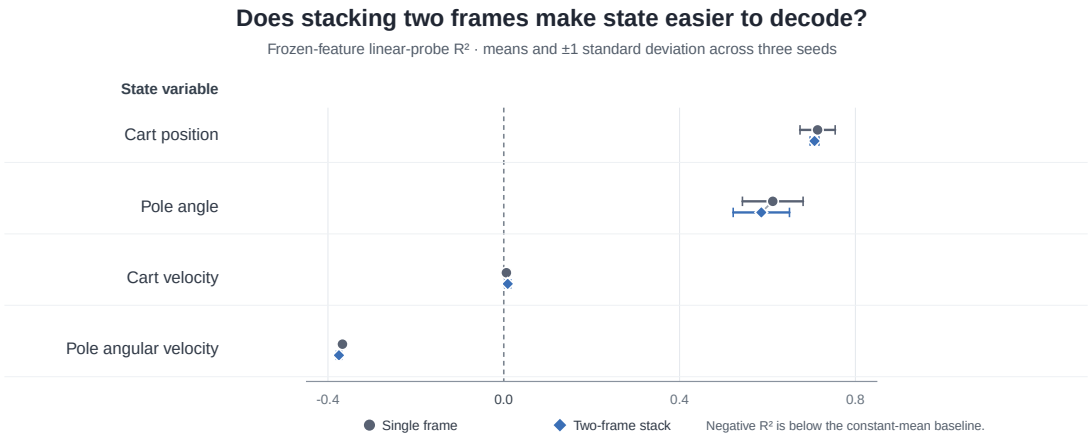


Figure 3.7 Single-frame and two-frame state probe comparison

The plot uses means and standard deviations across three seeds, with circular scoring for angular

targets. The complete measurements are available in the committed JSON results file.

The result is more limited than the hypothesis that a second frame will recover velocity. Position is more readily decoded than velocity from a single frame, but stacking two frames does not reliably improve the velocity measurements in this configuration. Cart-velocity R^2 increases by only about 0.002, while the pole-angular-velocity score becomes slightly worse. The position scores also do not improve.

This is a result about the tested image resolution, view transformations, encoder, data sampling, and training budget. It does not prove that velocity cannot be inferred from image sequences. It shows that simply providing two consecutive frames was not enough to make the tested velocities reliably available to this linear probe.

The main contrastive training run and this aliasing diagnostic also use different training budgets, so their results should not be treated as a controlled comparison with all other factors held fixed. In both cases, the probes use state labels only for evaluation. The contrastive objective itself still sees rendered frames and generated view pairs.

The larger lesson is the one that motivated this chapter. The encoder is optimized for the task stated by its loss. NT-Xent rewards representations that distinguish a frame's augmented views from views of other frames. That can organize representations and make source identity more accessible, but the loss does not explicitly require the physical state needed by a world model. To learn that state, the objective and the information available to the encoder matter.

The next section examines a failure that can occur even before asking which physical variables are represented: collapse, in which different inputs map to nearly identical representations and the model satisfies a similarity objective without learning a useful distinction.

3.4. The collapse problem and how it is avoided

Before comparing ways to prevent collapse, we need a concrete failure case: agreement between views can be achieved without retaining useful information.

3.4.1. Demonstrating collapse

Consider the simplest possible objective for two views of each frame: make their representations equal. Let $p_n^{(1)}$ and $p_n^{(2)}$ denote the vectors compared by the loss for the two views of frame n. For a batch of N frames and representations of dimension d, define

$$\frac{1}{Nd}\sum_{n=1}^{N} \| p_n^{(1)} - p_n^{(2)} \|_2^2.$$

This objective is nonnegative. It reaches zero whenever the two views produce identical

representations.

But it does not require different frames to produce different representations. Suppose the encoder returns the same constant vector for every input. Then every positive pair matches exactly, the loss is zero, and the objective has reached its global minimum. The solution works regardless of what the images contain. It does not need to represent cart position, pole angle, or any other aspect of the physical state.

This is complete collapse. All inputs map to one point.

There is also dimensional collapse. Here, the representation still varies across inputs, but its variation is restricted to only a few of the available dimensions. The encoder may produce a 32-dimensional vector while using only a small number of independent directions to distinguish observations. These are different failure modes. We therefore need more than one measurement to diagnose them.

Measuring collapse

The first measurement is spread: the standard deviation of each coordinate across a collection of representations, averaged over all d coordinates. If the encoder returns a constant vector for every input, the spread is zero. If its outputs become almost identical, the spread approaches zero.

Spread alone is not enough, however. A representation can use many independent directions while all its coordinates have very small values. Conversely, a representation can have large coordinate variation but vary mainly along one direction.

The second measurement is effective rank, which summarizes how many directions carry substantial variation.

Let $\lambda_1, \ldots, \lambda_d$ be the eigenvalues of the representation covariance matrix, and define the normalized eigenvalues

$$q_j = \frac{\lambda_j}{\sum_{\ell=1}^{d} \lambda_\ell}.$$

The effective rank is the exponential of the entropy of this distribution:

$$\exp\left(-\sum_{j=1}^{d} q_j \ln q_j \right).$$

If the variance is distributed equally across all d directions, the effective rank is d. If all variation lies in one direction, the effective rank is one. Intermediate values indicate that variation is distributed across a smaller number of directions.

The normalization removes overall scale. Multiplying every representation by 10^{-5} does not change the effective rank, because all covariance eigenvalues are multiplied by the same squared factor and the normalized q_j remain unchanged. This is useful for measuring dimensional collapse, but it means effective rank cannot detect a representation whose spread has shrunk almost to zero.

The following excerpt is from the diagnostic code in ch03/06_collapse_demo.py. It computes both measurements from a matrix of representations.

listing 3.6 summarizes the diagnostics used to distinguish low feature spread from low effective rank.

Listing 3.6 Feature spread and effective rank

```
centered = features - features.mean(axis=0)
s = np.linalg.svd(centered, compute_uv=False)
q = s**2 / (s**2).sum()
rank = np.exp(-(q * np.log(q + 1e-15)).sum())
spread = features.std(axis=0).mean()
```

Effective rank summarizes how broadly variance is distributed across feature directions; spread measures the average scale of individual coordinates. Effective rank is undefined when every singular value is zero, so the full implementation checks this degenerate case explicitly.

The full repository function also validates its input and handles the degenerate all-zero case. We use this diagnostic on the encoder output, not just the projection head, because the encoder output is the representation retained for later use.

What happens when we remove the negatives?

The repository's collapse demonstration compares two training conditions using the same convolutional encoder, projection head, data, optimizer, and view pipeline. The only intended difference is the objective: one run uses mean-squared attraction between positive views; the other uses NT-Xent.

The experiment uses cart-pole balance, 3,000 training frames sampled across episodes, batch size 64, 15 epochs, and three seeds. It measures the spread of the encoder output before training and after every epoch. The recorded results are stored in ch03/results/collapse_demo_metrics.json.

figure 3.8 plots the final encoder feature spread under attraction-only training and NT-Xent.

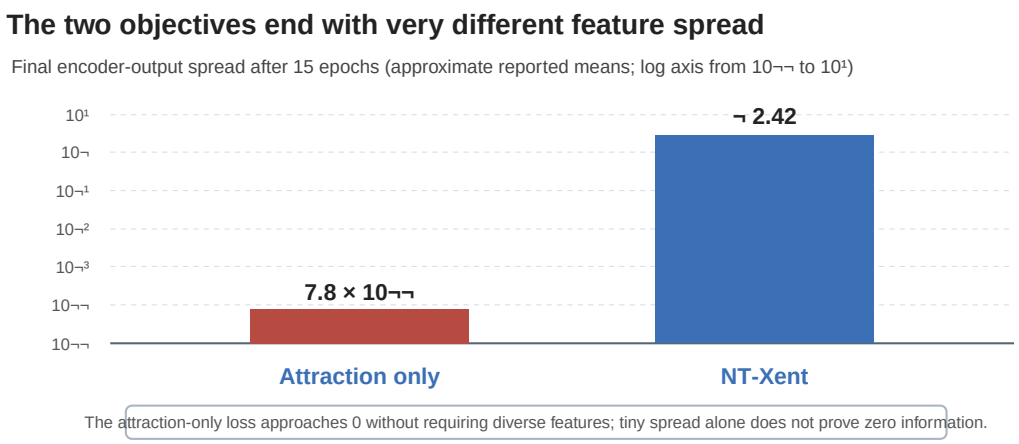


Figure 3.8 How representation collapse appears

Under attraction only, the encoder spread falls from approximately 5.2×10^{-4} at initialization to 7.8×10^{-5} after 15 epochs. The training loss falls to approximately 2.6×10^{-10}. Under NT-Xent, the encoder spread reaches approximately 2.42, while the final training loss is approximately 3.16. The two conditions behave very differently: attraction alone can achieve an almost-zero loss while allowing its already small representation variation to shrink further.

This finding needs a careful interpretation. The randomly initialized encoder already has a very small spread in this experiment. The run therefore does not demonstrate that a useful, well-trained representation was destroyed by attraction-only training. It demonstrates that the attraction loss does not require diversity and allows the representation to remain near-constant.

Effective rank gives a complementary result. Under attraction only, the final encoder effective rank remains about 12.4 out of 16, even as spread shrinks to roughly 10^{-5}. Under NT-Xent, effective rank drops sharply early in training and then recovers to about 7.0 by the final epoch. An effective-rank measurement alone would therefore have failed to identify the attraction-only representation's extremely small scale.

NOTE

A small spread is not, on its own, proof that a representation contains no information. A representation can encode distinctions at a small scale. The recorded experiment does not probe the attraction-only encoder for physical-state information, so we do not claim that its tiny variation is necessarily useless. The result is narrower: the objective does not demand spread or prevent a constant representation from minimizing the loss.

Why does NT-Xent behave differently?

If all views map to the same nonzero vector, every cosine similarity is one. The positive and the negatives then have identical scores, so the probability assigned to the positive is $1/(2N-1)$ and

the loss is $\ln(2N-1)$. The encoder cannot reduce the loss by distinguishing the positive from the negatives, because it has collapsed them all together. NT-Xent rewards a lower loss when the positive receives a higher score than the negatives.

The negative pairs therefore give contrastive learning a mechanism that attraction alone lacks. But they bring their own design choices, including the number and quality of the negatives. The next section examines alternatives that do not require them.

3.4.2. Three mechanisms in brief

SimSiam, BYOL, and Barlow Twins use different mechanisms to avoid collapse without contrastive negative pairs. Their shared starting point is that two views of the same frame should produce related representations. What differs is how each method prevents that requirement from being satisfied by an uninformative constant output.

The repository implements reduced-scale versions of all three methods. The study uses cart-pole frames, five random seeds, 30 epochs, up to 6,000 training frames, batch size 128, and Adam with learning rate 10^{-3}. The encoder output has 32 dimensions, while the projection and predictor heads use 128-dimensional embeddings. These experiments examine the core mechanisms at small scale; they are not exact reproductions of the large-scale ImageNet experiments in the original papers.

Different methods prevent collapse in different ways, and their mechanisms are not interchangeable. table 3.2 compares their design choices and limitations; the reduced-scale experiment then compares feature statistics and frozen-feature probes.

Table 3.2 Three mechanisms used to avoid collapse

Method	**Anti-collapse mechanism**	**Important limitation**
SimSiam	Stop-gradient target branch, predictor, and normalization.	Asymmetry alone does not guarantee a useful, non-collapsed state.
BYOL	Exponential-moving-average target encoder.	Success still depends on the full training setup.
Barlow Twins	Match correlation diagonals; penalize off-diagonal redundancy.	Decorrelation does not ensure physical-state sufficiency.
VICReg	Invariance plus variance floor and covariance penalty.	Feature constraints do not guarantee state sufficiency.

The reduced-scale cart-pole comparison uses five seeds. Mean frozen-feature probe scores are 0.266 for SimSiam, 0.268 for BYOL, and 0.285 for Barlow Twins. Their effective ranks out of 32 dimensions are 4.23, 7.07, and 11.18, while average coordinate spreads are about 1.93, 0.306, and

0.0616, respectively. The methods produce similar mean probe scores but noticeably different feature statistics. These measurements do not establish a universal ranking: the runs use small models and data, and do not isolate the objective from every architectural choice.

The feature-spread diagnostic appears in figure 3.8; figure 3.9 summarizes the complementary anti-collapse mechanisms.

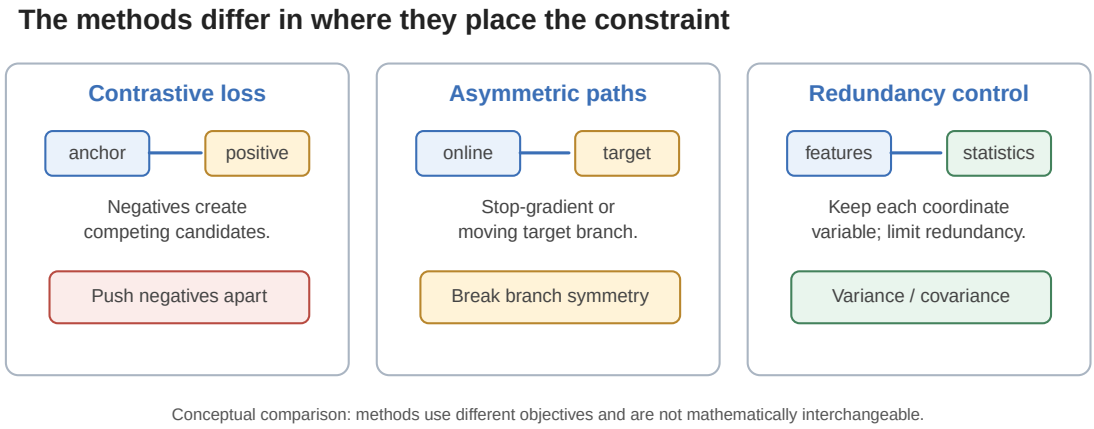


Figure 3.9 Different methods prevent collapse in different ways

3.4.3. VICReg: Making the anti-collapse force explicit

VICReg stands for variance-invariance-covariance regularization. It receives two batches of embeddings, Z^A and Z^B, each with N rows and d coordinates. It combines three terms, each with a distinct purpose.

The variance term

The variance term sets a floor on the standard deviation of each coordinate across a batch. Let $Z_{:,j}$ denote column j, and let S be its regularized standard deviation:

$$S(u) = \sqrt{(u) + \epsilon}.$$

The variance hinge is

$$v(Z) = \frac{1}{d}\sum_{j=1}^{d} \max\left(0, \gamma - S(Z_{:,j})\right)$$

Here, γ is the minimum target standard deviation and $\epsilon>0$ stabilizes the square root. The repository uses $\gamma = 1$ and $\epsilon = 10^{-4}$.

The hinge is zero for a coordinate whose standard deviation is already at least γ. A coordinate below the threshold incurs a penalty proportional to the shortfall. The term therefore sets a floor; it

does not reward arbitrarily increasing a coordinate's scale.

The small ϵ matters near zero variance. Without it, differentiating the square root at zero can produce an undefined or unbounded derivative. With $\epsilon = 10^{-4}$, the derivative of $\sqrt{v+\epsilon}$ with respect to variance is bounded above by $1/(2\sqrt{\epsilon}) = 50$.

The invariance term

The invariance term asks the two views to produce similar embeddings. In the implementation, it is the mean squared error between matching rows:

$$s(Z^A, Z^B) = \frac{1}{Nd} \sum_{n=1}^{N} \| z_n^A - z_n^B \|_2^2$$

This is the positive-view attraction from the collapse demonstration, but now it is only one part of the objective. By itself, it would allow a constant representation. In VICReg, the variance and covariance terms constrain what solutions can minimize the total loss.

The covariance term

The covariance term discourages redundancy between different coordinates. Let $\bar{z}$ be the mean row of the batch. Define the sample covariance matrix

$$C(Z) = \frac{1}{N-1} \sum_{n=1}^{N} (z_n - \bar{z})(z_n - \bar{z})^\top.$$

Its diagonal contains the coordinate variances. Its off-diagonal entries contain the covariances between different coordinates. The VICReg covariance term penalizes squared off-diagonal entries:

$$c(Z) = \frac{1}{d} \sum_{i \neq j} [C(Z)]_{ij}^2$$

If all off-diagonal entries are zero, the coordinates are pairwise uncorrelated in the batch. This reduces second-order redundancy, but it does not establish that the coordinates are statistically independent: nonlinear dependence can remain even when covariance is zero.

The total VICReg loss is a weighted sum

The objective combines invariance, variance, and covariance penalties using coefficients that set their relative contributions:

$$
\begin{aligned}
L_{\text{VICReg}} = \; & \lambda s(Z^A, Z^B) \\
& + \mu\big[v(Z^A) + v(Z^B)\big] \\
& + \nu\big[c(Z^A) + c(Z^B)\big].
\end{aligned}
$$

The implementation uses $\lambda = 25$, $\mu = 25$, and $\nu = 1$. These coefficients set the relative influence of invariance, variance, and covariance penalties. Both embedding batches receive their own variance and covariance penalties.

Consider what the three terms do to degenerate solutions.

A constant representation. If every coordinate is constant across the batch, the invariance term is zero and the covariance term is zero. But the regularized standard deviation of each coordinate is only $\sqrt{\epsilon} = 0.01$. With $\gamma = 1$, each coordinate incurs a variance-hinge penalty of 0.99. Across both views, the weighted variance term adds $25 \times 2 \times 0.99 = 49.5$ to the total loss. Collapse is no longer a zero-cost solution.

A single repeated direction. Suppose all coordinates are identical copies of one unit-variance scalar. The views can match and every coordinate meets the variance floor, so the invariance and variance terms can both be zero. But every off-diagonal covariance equals one. With $d = 16$, there are $16 \times 15 = 240$ off-diagonal entries, and the covariance cost for each batch is $240/16 = 15$. Across the two views, the covariance term contributes 30. This discourages dimensional collapse into one repeated direction.

Unrelated but varied embeddings. A model could produce embeddings with sufficient variance and little coordinate covariance without making the two views of the same frame agree. The variance and covariance terms would not necessarily penalize this configuration, but the invariance term would. It is the term that ties the representations of paired views together.

These examples show why no one of the three terms replaces the others. Variance prevents coordinates from becoming constant, covariance discourages redundancy, and invariance links representations of positive views. Their combination makes the constraints explicit, instead of relying solely on the behavior of the training procedure.

A compact implementation keeps the three penalties explicit:

```
def vicreg_loss(za, zb, sim_coeff=25.0, std_coeff=25.0,
                cov_coeff=1.0, gamma=1.0, eps=1e-4):
    n, d = za.shape
    sim = F.mse_loss(za, zb)                                #A

    def variance(z):
        v = z.var(dim=0, unbiased=False)
        std = torch.sqrt(v + eps)                           #B
        return F.relu(gamma - std).mean()                   #C

    def covariance(z):
```

```
        z = z - z.mean(dim=0)
        cov = (z.T @ z) / (n - 1)                          #D
        off = ~torch.eye(d, dtype=torch.bool, device=z.device)
        return cov[off].pow(2).sum() / d                    #E

    var = variance(za) + variance(zb)
    cov = covariance(za) + covariance(zb)
    total = sim_coeff * sim + std_coeff * var + cov_coeff * cov
    return total, sim, var, cov
```

A Computes the invariance term from equation 3.10.

B Computes the regularized standard deviation used by equation 3.9.

C Penalizes coordinates whose standard deviation is below γ.

D Computes the batch covariance matrix.

E Sums the squared off-diagonal entries, divided by d.

The variance calculation uses population variance, while the covariance estimate divides by (N-1), matching the repository implementation. The function returns the total loss and each of its three components so the training script can record their behavior separately.

Training VICReg on cart-pole frames

The standalone VICReg runner is ch03/08_vicreg.py. It uses the convolutional encoder and projection head from Section 3.3.4, with 4,000 training frames, batch size 64, 15 epochs, Adam at learning rate 10^{-3}, and three seeds. The three loss terms operate on the 16-dimensional projection-head outputs; the downstream state probe reads the encoder output before the projection head.

The recorded VICReg curves in figure 3.10 show the total loss and three unweighted loss components. The variance hinge falls close to zero by around epoch five, while the covariance term rises early and then declines. The invariance term remains comparatively small. This pattern is consistent with the variance floor being established before the covariance penalty reduces redundancy, though the experiment does not separately test that causal interpretation.

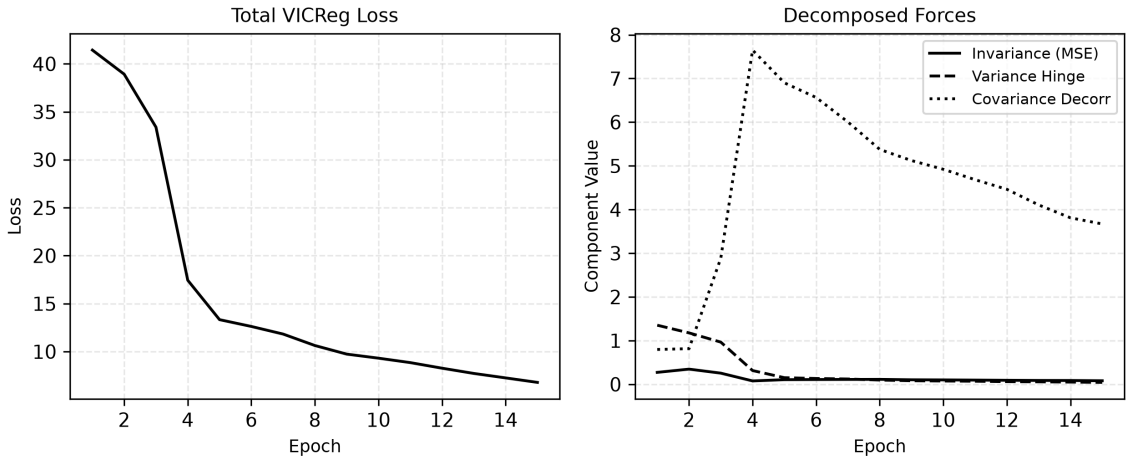


Figure 3.10 The three terms in VICReg

In this standalone configuration, the state probe reports mean $R^2 = 0.248 \pm 0.005$ across three seeds. Cart position and pole angle are more linearly recoverable than the velocities. These results describe this run's architecture, sample cap, training budget, and probe. The contrastive training run in Section 3.3 uses a different budget and number of training frames, so these scores should not be interpreted as a controlled comparison of objectives. The objective-comparison study later in Section 3.5 uses a separately specified protocol.

VICReg avoids explicit negative pairs, but that does not make it free of design choices. The covariance matrix is estimated from a finite batch, and the coefficients λ, μ, ν determine the balance between the three terms. This chapter uses the stated coefficients rather than tuning them to obtain a preferred result.

The methods address collapse through different mechanisms: NT-Xent uses negative pairs, SimSiam and BYOL use asymmetric training paths, Barlow Twins reduces cross-correlation, and VICReg imposes explicit variance and covariance constraints. None of these mechanisms alone establishes that the representation preserves all information needed for prediction. Masked prediction uses a different signal: it reconstructs parts of the observation that were hidden from the encoder.

3.5. Masked prediction

Rather than matching views or using a past context to identify a future, masked prediction asks a model to infer hidden parts of an observation from the visible remainder.

3.5.1. From masked words to masked patches

BERT trains a language model by hiding selected tokens and asking it to predict them from the surrounding context. The targets are the identities of the missing tokens, not a human-written label for the whole sentence. The model learns from the structure already present in the text.

A masked autoencoder MAE applies a related idea to images. It divides an image into patches, hides a random subset, and predicts the missing pixel values from the visible patches. The training target comes directly from the input image. No class label, action, reward, or physics state is required.

The masking ratio matters. If only a few patches are hidden, nearby visible pixels may make reconstruction relatively easy. Hiding a large fraction of the image creates a more demanding prediction task: the model must use the remaining patches and their relationships to reconstruct the missing content. The original MAE work found that a high masking ratio, such as 75%, could produce a useful self-supervised task at scale. That result motivates our choice; it does not establish that 75% is optimal for every dataset or architecture.

Our experiment uses 64×64 RGB frames from the Control Suite's Cartpole Balance task. Each image is divided into an 8×8 grid, giving 64 patches. With a masking ratio of 75%, the encoder sees 16 randomly selected patches during training. The decoder predicts all 64 patches, but only the masked patches contribute to the loss.

This is an asymmetric architecture. The encoder processes the visible patches without carrying a placeholder token for every hidden patch. The decoder receives the encoded visible patches, inserts learned mask tokens into the missing positions, restores the original order, and predicts the pixel values. This division keeps the encoder's training input smaller than the full image's patch sequence.

There is an important boundary to the analogy with language. A rendered physics frame is far simpler than a natural image, and our images contain a stable background. Reconstructing a Cartpole frame is not the same as reconstructing the physical state that generated it. In particular, a single image need not reveal velocity. The experiment tests what this compact MAE learns under its particular data, architecture, and training budget.

3.5.2. Building a masked autoencoder

The first operation is to turn an image into a sequence of patches. For a batch of images, the resulting tensor has shape

$$P \in \mathbb{R}^{B \times N \times D},$$

where B is the batch size, $N = 64$ is the number of patches, and $D = 8 \times 8 \times 3 = 192$ is the number of pixel-channel values in each flattened patch.

A learned linear layer embeds each patch into a 64-dimensional token. Positional embeddings distinguish the patches' locations. The model then samples a different random mask for each image in the batch. It retains 16 tokens and sends only those tokens through the encoder.

For readability, this excerpt omits patchification and decoder helpers; the canonical implementation supplies both.

listing 3.7 isolates the visible-patch encoder and masked-patch reconstruction path.

Listing 3.7 The masked-prediction path (schematic)

```
visible, restore_ids, mask = random_mask(
    patch_tokens, keep_ratio=0.25
)
encoded = encoder(visible)
predicted = decoder(encoded, restore_ids)
loss = masked_mse(patch_targets, predicted, mask)
```

This schematic omits patch indexing and decoder details. The defining choices are that the encoder processes only visible patches and that reconstruction error is computed only for the hidden patches. The complete implementation handles masking, positional order, and the reconstruction target.

The random values in noise define a permutation of the patch positions. ids_keep selects the visible tokens, while ids_restore records how to put the visible tokens and mask tokens back into their original order for decoding. The mask uses 1 for a hidden patch and 0 for a visible patch. These details are important: a decoder must know which predicted patch belongs in which image location.

The training objective is the mean squared error over masked patches only. Let P_{bi} denote patch i from image b, let $\hat{P}_{bi}$ be its prediction, and let M_{bi} equal 1 if that patch was masked and 0 otherwise. With D pixel-channel values per patch, the loss is

$$\frac{\sum_{b=1}^{B}\sum_{i=1}^{N} M_{bi}\left[\frac{1}{D}\sum_{j=1}^{D}(P_{bij}-\hat{P}_{bij})^2\right]}{\sum_{b=1}^{B}\sum_{i=1}^{N} M_{bi}}.$$

The denominator counts masked patches, so each hidden patch contributes its mean pixel-channel error. Visible patches do not contribute to the loss, even though the decoder produces predictions for them too. This matches the implementation in masked_mse_loss.

The physics states collected from the simulator do not enter this objective. They remain available for evaluation, including the linear-probe tests used elsewhere in the chapter. The only training target here is the hidden image content.

figure 3.11 pairs the masked-autoencoder data path with recorded examples of original frames, masked inputs, and reconstructions.

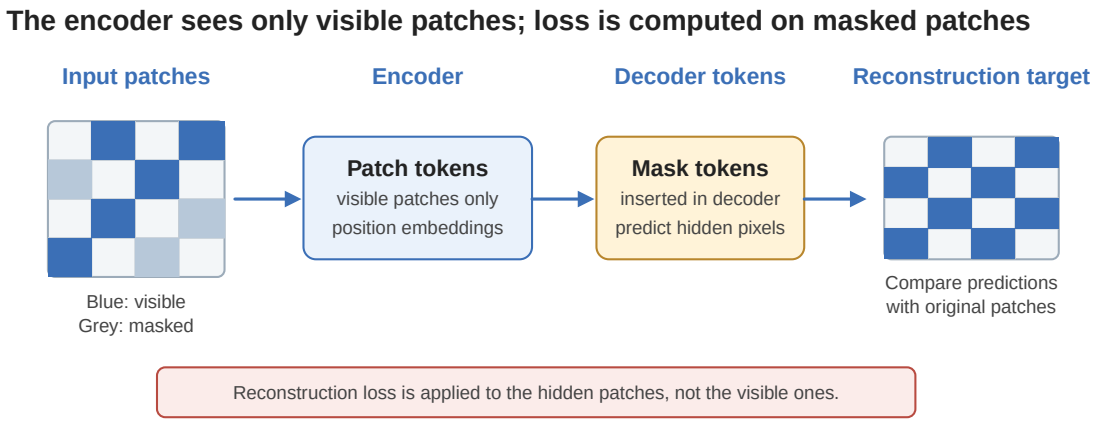
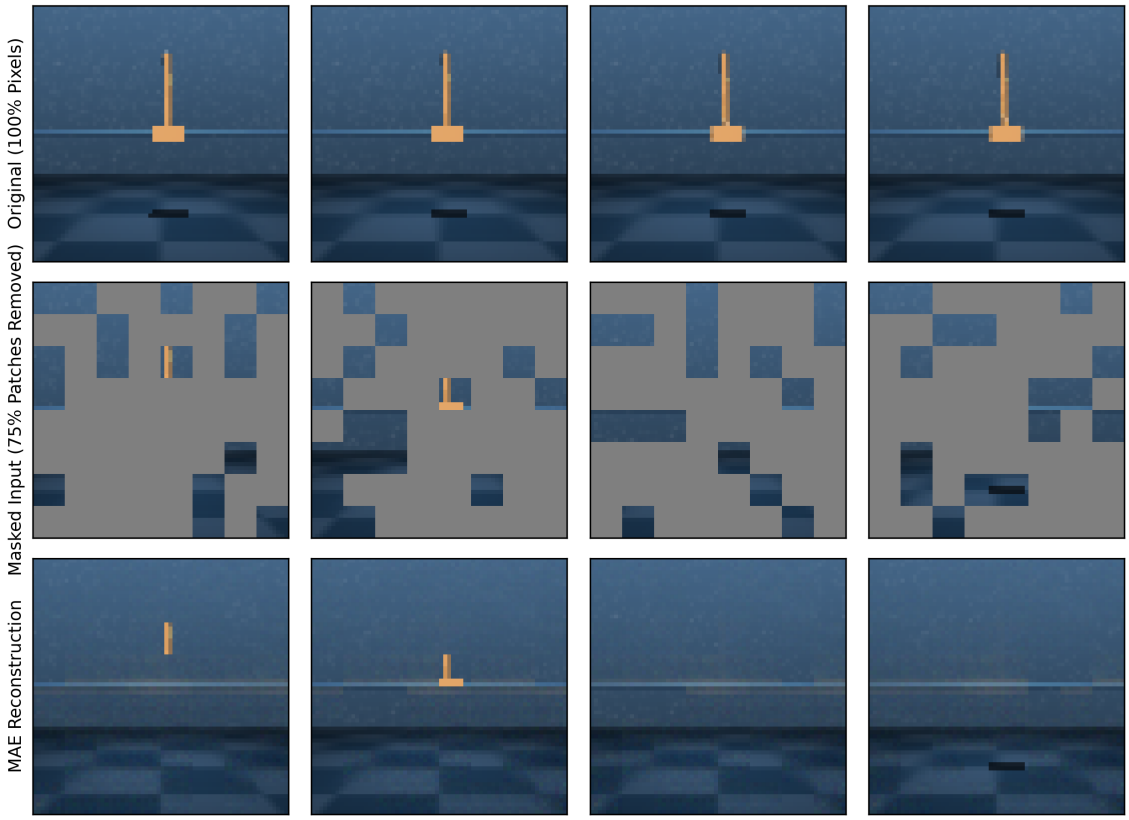


Figure 3.11 Visible patches, latent encoding, and masked-patch reconstruction



The key architectural distinction is that the encoder sees only visible patches; the decoder inserts mask tokens and predicts the hidden pixels. Only the hidden patches contribute to the reconstruction loss.

For evaluation, the experiment uses a 64-dimensional feature vector formed by mean-pooling the encoder's patch tokens. The probe is applied to this frozen representation, not to the decoder's pixel predictions.

Variational autoencoders and generative adversarial networks use additional objectives and trade-

offs, but are outside the comparison here.

3.5.3. Reconstruction versus representation

A reconstruction loss measures how well the model rebuilds its input. A state probe asks a different question: how much information about the simulator's state can be recovered by a linear map from the learned representation? Good reconstruction does not guarantee a strong state representation, just as a strong state probe does not guarantee pixel-accurate reconstruction.

Consider the target in equation 3.13. The error is measured across the pixels of every masked patch, including patches containing the floor and background. These pixels are part of the image and therefore part of the objective, even when their appearance says little about the cart's position or the pole's angular velocity. The encoder can be rewarded for retaining information that helps reconstruct pixels without being equally rewarded for making every physical variable easy to decode linearly.

This is a hypothesis about the objective, not a conclusion that follows from the equation alone. We test it by examining reconstructions and measuring frozen-feature linear probes. The interpretation must remain bounded by the experiment: Cartpole frames at 64×64, the current patch-transformer architecture, a 75% masking ratio, and the stated training budget. These frames are far simpler than natural video, so this comparison does not establish a general ranking of reconstruction and contrastive methods.

The probe results. The standalone MAE experiment uses seeds 0, 1, and 2, 4,000 episode-stratified training frames, 1,000 validation frames, 15 epochs, a batch size of 64, and Adam with learning rate 10^{-3}. Its probe uses the mean-pooled 64-dimensional encoder tokens. Across the three training seeds, the mean R^2 across the four Cartpole state variables is 0.170 ± 0.004. This is the result recorded in ch03/results/mae_metrics.json.

figure 3.12 compares complete configurations, not loss functions in isolation. All four methods use three seeds, 12 epochs, a batch size of 64, 3,500 training frames, and 1,000 validation frames. Error bars show the standard deviation across trained seeds, not across independently collected environments.

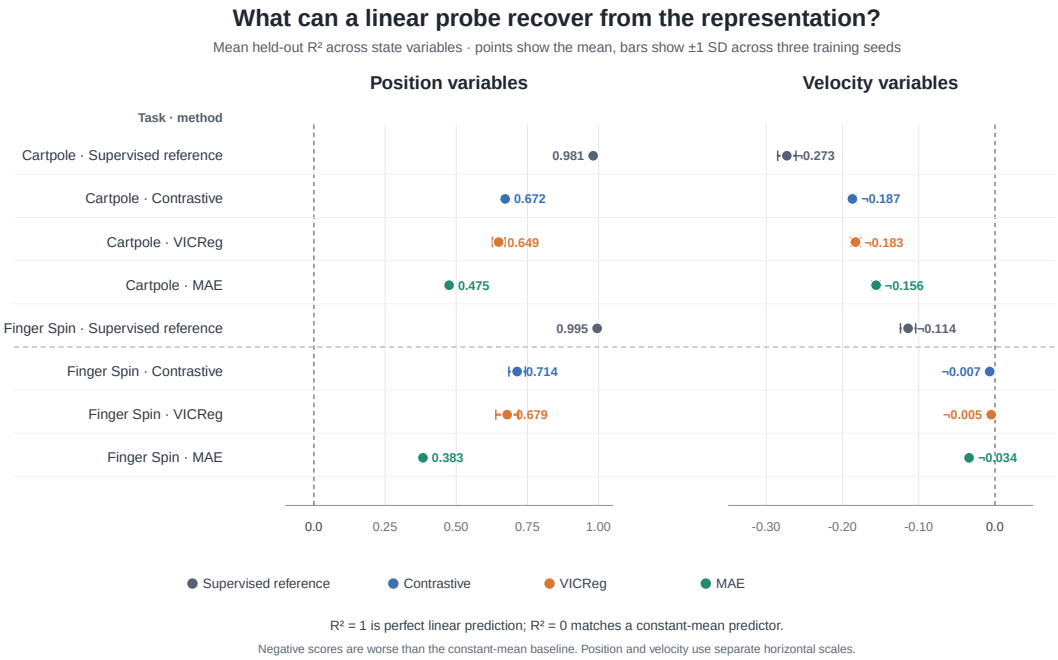


Figure 3.12 State-probe comparison across method families

The figure compares complete model configurations, not objective functions in isolation. Each method uses seeds 0, 1, and 2, 12 epochs, a batch size of 64, a learning rate of 10^{-3}, 3,500 training frames, and 1,000 validation frames. Error bars show standard deviation across the three trained seeds, not uncertainty across independently collected environments. The Per-variable scores are available in the committed JSON results file; they are omitted here to keep the main comparison readable.

For Finger Spin, the absolute spinner hinge angle is excluded from the aggregate position score. Its rendered geometry has π-periodic symmetry, so a static image cannot uniquely identify its absolute 2π angle. The position summary therefore averages the other position variables, and the velocity summary averages the task's velocity variables. The exclusion is part of the metric definition, not a discarded unsuccessful prediction.

The supervised row has a specific interpretation. The encoder and a prediction head were trained using privileged simulator-state labels, after which a separate linear probe was fit to frozen encoder features. The reported numbers belong to that separate probe, not to the trained prediction head's held-out score. The supervised reference is not a self-supervised method.

The per-variable results make the aggregates easier to interpret. The entries are mean R^2 ± standard deviation across the same three seeds and follow the state-vector order: Cartpole uses qpos[0], qpos[1], qvel[0], qvel[1]. Finger Spin uses qpos[0], qpos[1], qpos[2], qvel[0], qvel[1], qvel[2]. For Finger Spin qpos[2], the table reports a separate π-periodic score for visible orientation; absolute 2π angle remains unidentifiable from a static render. That score is displayed for completeness but

excluded from the aggregate position score.

R^2 values below zero mean that the probe performs worse on the validation data than a constant predictor using the validation-target mean. In these runs, velocity is difficult to decode linearly across methods. The MAE's Cartpole position score is below those of the contrastive and VICReg representations in this comparison, while its Cartpole velocity score is less negative. On Finger Spin, MAE performs poorly on the second position variable and has negative velocity scores. No single aggregate tells the whole story.

These observations do not isolate the objective as the cause. MAE uses a patch-transformer encoder and decoder, and its evaluated feature vector is 64-dimensional. The contrastive and VICReg representations in this comparison use a convolutional backbone with a 16-dimensional latent representation. The supervised reference additionally uses privileged state labels during pretraining. figure 3.12 is therefore a method-family comparison under a stated budget, not a loss-only ablation. The measurements describe these complete configurations; they do not establish that masked reconstruction is inherently inferior to contrastive learning or VICReg.

figure 3.13 contrasts a model trained to predict a latent representation with one trained to reconstruct pixel values.

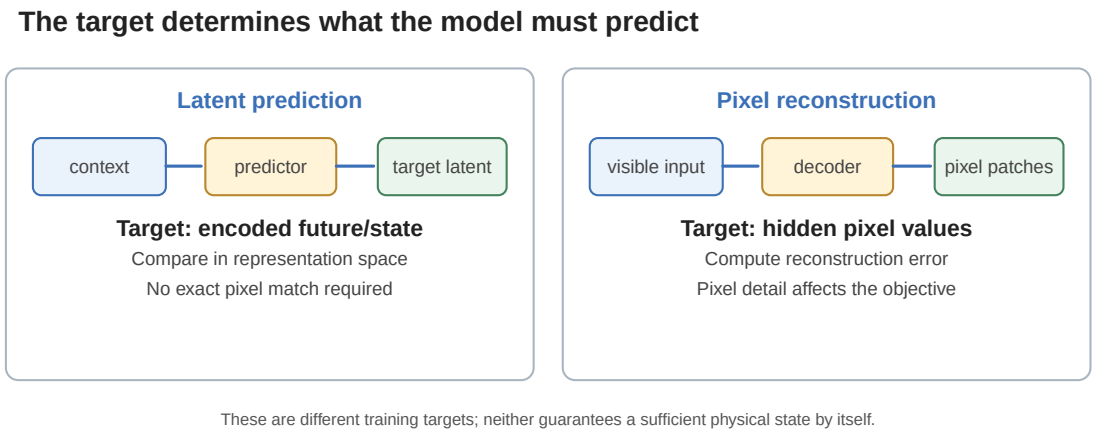


Figure 3.13 Predicting a representation is not the same as reconstructing pixels

The result illustrates the central distinction: a training objective encourages the encoder to preserve information that helps solve its target. Masked prediction is useful when reconstructing hidden observations matters, but pixel reconstruction alone does not ensure that physical state variables will be easy to decode. A representation intended for a world model must also support prediction.

Before predicting in a representation space, we should ask how much representational capacity the state needs. The next section studies that question through latent bottlenecks.

3.6. Latent bottlenecks and compact state

A representation also has a capacity constraint: increasing its dimension does not guarantee that the additional coordinates retain useful information about the task.

3.6.1. The information bottleneck as a lens

Let X denote an observation, Z its learned representation, and Y a task-relevant variable. For a world model, X might be an image and Y might describe the physical state needed to predict what happens next.

The information bottleneck asks for a representation that preserves information relevant to Y while discarding as much unnecessary information about X as possible. One way to express this trade-off is

$$\mathcal{L}_{\mathrm{IB}} = I(Z; X) - \beta I(Z; Y).$$

Here, $I(A; B)$ is the mutual information between random variables A and B. It measures statistical dependence: it is zero when the variables are independent and increases as knowing one variable provides more information about the other. The coefficient β controls the exchange between compression and relevance.

The two terms pull in opposite directions.

Compression. Minimizing $I(Z; X)$ discourages the representation from carrying information about every detail of the input.

Relevance. Minimizing the full loss rewards a large $I(Z; Y)$, because that term is subtracted. The representation should preserve information that helps distinguish task-relevant outcomes.

When β is small, compression receives more weight relative to relevance. As β increases, preserving information about Y becomes more important. The best trade-off depends on which information the task actually needs.

This gives a useful interpretation of a compact state. If an image contains a cart, a pole, a floor, lighting, and background details, the agent may not need to encode all those details equally. It needs enough information to distinguish states that matter for prediction and control. But compression must stop before two physically different situations that require different decisions become indistinguishable in the representation.

A representation needs to preserve enough information for the task without carrying unnecessary detail. A bottleneck can help limit irrelevant detail, but an overly narrow representation can also discard information needed for prediction.

There is an important distinction between this information-theoretic objective and simply choosing

a small vector. A deterministic encoder with a low-dimensional continuous output is not, by that fact alone, optimizing $I(Z; X)$. In formal information-bottleneck formulations, the relationship between input and representation is treated probabilistically, and the mutual-information terms must be defined and optimized or bounded appropriately. A deterministic autoencoder with a narrow latent layer imposes a practical capacity constraint; it does not automatically implement equation 3.14.

The same distinction applies to VICReg. Its invariance, variance, and covariance penalties encourage particular properties of learned embeddings, but VICReg does not optimize the information-bottleneck objective in equation 3.14. The equation is a conceptual lens for understanding what we want from a representation, not a claim that every self-supervised objective is secretly an information-bottleneck algorithm.

In this chapter's experiments, the simulator's physics state remains an evaluation target. The contrastive loss does not receive that state as a training label, and the latent-dimension sweep does not calculate mutual information directly. We use a linear probe to measure one concrete property: how much of the state is linearly recoverable from the learned features.

3.6.2. Choosing the latent dimension

The practical question is how many dimensions the encoder needs. It is tempting to set the latent dimension equal to the number of physical state variables. That is not a reliable rule.

A physical variable need not map to one latent coordinate. A learned feature can combine several state variables, and a single variable may require several features to become easy to decode. This is especially relevant to angles: a linear probe can have difficulty reading an angle directly near the wraparound boundary, so the evaluation uses periodic encoding and a circular R^2 score for angular positions.

Nor does a larger latent dimension automatically make the representation better. The encoder may fail to learn useful features within the available training budget, or its additional coordinates may not improve the information that the probe can recover. The dimension is a capacity choice; the probe is a measurement of the result.

The hypothesis

The benchmark supplies two tasks with different physical-state dimensions:

Cartpole Balance: four state variables.

Cheetah Run: eighteen state variables.

The hypothesis is that Cartpole will reach a useful probe-performance plateau at a smaller latent

dimension, while Cheetah will benefit from larger representations for longer. At $d = 2$, Cartpole's representation may be too restrictive to preserve the variation needed to distinguish its physical states.

This hypothesis concerns the performance of the complete learned representation, not just the arithmetic difference between the number of latent dimensions and the number of state variables. A four-variable system does not imply that a four-dimensional latent is sufficient, nor that four is the optimal dimension for a linear probe.

The experiment

The repository's ch03/10_bottleneck_sweep.py trains a separate contrastive encoder for every combination of task, latent dimension, and random seed. The dimensions are $d \in 2, 4, 8, 16, 64, 256$, with seeds 0, 1, and 2. Each run uses the convolutional encoder from Section 3.3, with a projection head whose output dimension matches d.

The run uses Adam with a learning rate of 10^{-3}, a batch size of 64, and 10 training epochs. It selects 3,000 training frames and 1,000 validation frames, spreading the selected indices across episodes rather than taking a single prefix of each dataset. The contrastive objective operates on two augmented views of each frame. Once training is complete, the linear probe is fitted to the frozen encoder features, not the projection head's output.

listing 3.8 reduces the latent-dimension sweep to its essential training-and-probe steps.

Listing 3.8 The logic of a latent-dimension sweep (schematic)

```
for d in [2, 4, 8, 16, 64, 256]:
    encoder = train_encoder(latent_dim=d)
    features = encoder(validation_images)
    score = linear_probe_r2(features, validation_state)
    record(d, score)
```

This sketch isolates the comparison: change the representation dimension, train under the same protocol, and measure held-out state decodability. The runner contains the full training and evaluation procedures.

The sweep measures held-out linear-probe R^2, averaged across the task's physical-state variables. The probe standardizes frozen latent features before fitting a ridge-regression readout. Its angular-state evaluation uses periodic handling, so the score is not distorted merely because an angle crosses a wraparound boundary. The reported spread is the standard deviation across the three trained seeds, not across independently collected environments.

The measured result

The recorded sweep output is shown in figure 3.14.

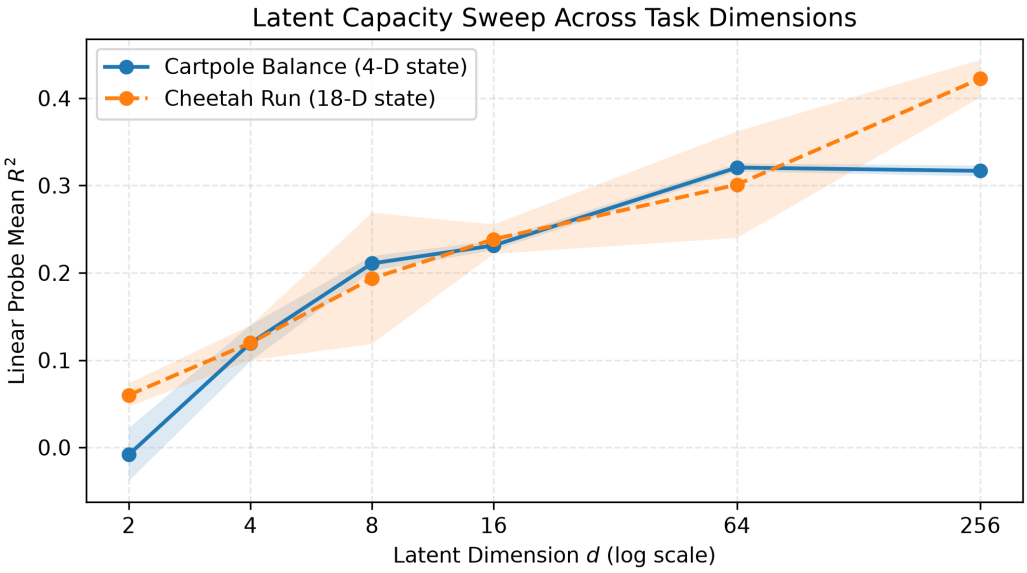


Figure 3.14 State-probe performance across latent dimensions

For Cartpole Balance, the mean probe R^2 begins at −0.008 for $d = 2$, rises to 0.119 at $d = 4$, and reaches 0.231 at $d = 16$. The strongest result in this sweep is 0.321 ± 0.005 at $d = 64$. Increasing the dimension to $d = 256$ does not improve it: the result is 0.317 ± 0.006. Under this configuration, performance has largely levelled off between 64 and 256 dimensions.

The result does not place the plateau at the task's four state variables. A four-dimensional output provides four real-valued coordinates, but that alone does not mean the encoder has organized those coordinates into a representation from which the physical state is easy to decode. At $d = 4$, the probe R^2 of 0.119 remains weak. The poor score at $d = 2$ is consistent with a restrictive representation losing useful state information, but this experiment does not explicitly identify pairs of physical states that have collided in latent space. The score alone is evidence of weak linear decodability, not a direct demonstration of a particular aliasing pair.

Cheetah Run behaves differently. Its mean probe R^2 rises from 0.060 ± 0.013 at $d = 2$, to 0.238 ± 0.017 at $d = 16$, and 0.301 ± 0.061 at $d = 64$. At $d = 256$, it reaches 0.422 ± 0.021, the best score measured for this task. Unlike Cartpole, Cheetah has not reached a clear plateau within the tested range.

These results support the direction of the hypothesis: the four-variable task approaches a plateau at a smaller dimension, while the eighteen-variable task continues to benefit from increased capacity through $d = 256$. They do not establish the location of Cheetah's eventual plateau. The sweep tests six dimensions, and its highest tested value is not necessarily sufficient.

The experiment is also not a controlled comparison of latent dimensions in which all else is mathematically identical. Each point comes from a newly trained encoder and projection head. The result therefore reflects how the fixed architecture, contrastive objective, data, and training budget

work at each output dimension. The evidence answers a practical question about this training setup, not a universal law connecting physical-state dimension to optimal latent size.

TIP

Choose the smallest dimension that has saturated. Start with a sweep rather than selecting d by intuition. Look for the smallest dimension at which the measured state probe stops improving meaningfully, and include variation across seeds. If performance is still rising at the largest tested dimension, the sweep has not yet identified a plateau.

The bottleneck in one paragraph

The latent dimension limits the coordinates available to the representation. Too little capacity can make task-relevant distinctions difficult to preserve; more capacity can improve decodability, but also costs computation and may not help the task. The right dimension is not automatically the state-vector length. It is the smallest practical representation that retains enough information for the work the agent must do, judged by measurements relevant to that work.

The distinction between reconstruction, invariance, and predictive information now becomes more concrete. Section 3.5 showed that reconstructing pixels does not guarantee that state is linearly available. This section shows that even a representation with more dimensions is useful only to the extent that it preserves the distinctions a probe can recover. In the next section, the question changes from which static observations should agree to which future observation follows from the past.

3.7. Predicting over time: From invariance to prediction

Matching two views is useful when their differences are irrelevant, whereas prediction requires retaining information about how the scene can change.

3.7.1. Why the future is not just another view

Return to the view-generation decision from Section 3.3. Two augmented versions of the same frame are positive examples because the objective asks the encoder to preserve information that survives those augmentations. The brightness and small crop changes are intended to be nuisance variations, so the learned representation should be relatively insensitive to them.

Now suppose we instead pair frames from times t and $t + \Delta$ as positives and ask their embeddings to agree. The scene may have changed during that interval. The cart may have moved, the pole may have rotated, and the object's velocity may have changed. An objective that rewards

agreement between the two embeddings can treat these differences as information to suppress.

That may be undesirable for a world model. The future depends on the current situation and its dynamics. If the representation is encouraged to remove the differences that signal those dynamics, a downstream predictor has less useful information to work with.

This is not a claim that temporal contrastive learning must always discard dynamics. The result depends on how temporal positives are constructed and what else the objective asks the model to preserve. The narrower point is that invariance and prediction impose different requirements. Invariance says which observations should agree; prediction says which future observations are compatible with a given past.

Our measurements from the earlier sections make this distinction concrete. figure 3.7 showed that velocity was poorly decoded from the learned single-frame representation under the tested configuration. Stacking two frames did not reliably fix the problem under those same experimental conditions. These results do not prove that velocity is fundamentally absent from images or impossible to recover from short sequences. They show that the representation and training setup did not make it reliably linearly available.

A frame-level invariance objective has no direct reason to preserve velocity. A temporal objective can instead reward a context for retaining information that helps identify a future observation. CPC is one way to implement that change.

3.7.2. Contrastive predictive coding

CPC, introduced by van den Oord and colleagues, learns by predicting future representations rather than reconstructing future pixels. It encodes a sequence of observations into latent vectors, summarizes the available past in a context vector, and trains that context to identify the correct future latent among other candidates.

Let a sequence of observations be $x_1, x_2, \ldots, x_T$. A shared encoder f_θ maps each observation to a latent representation:

$$z_t = f_\theta(x_t).$$

A recurrent model summarizes the representations observed up to time t:

$$c_t = g(z_1, z_2, \ldots, z_t).$$

The context c_t may depend on the current representation and every earlier representation in the sequence. It must not depend on future representations $z_{t+1}, z_{t+2}, \ldots$. Otherwise, the task would leak information from the answer into the context used to select that answer.

We use a gated recurrent unit GRU for g. A GRU maintains a learned hidden state as it processes

the sequence one step at a time. Its purpose here is to produce a compact context vector, not to guarantee that every relevant aspect of the past is retained. Whether the learned context retains useful information is an empirical question.

Scoring a candidate future

For each prediction horizon k, CPC learns a matrix W_k. It scores a candidate future latent using a bilinear compatibility function:

$$\exp\left(z_{t+k}^{\top} W_k c_t\right).$$

Here, z_{t+k} is the candidate representation at a future time, c_t is the context available at time t, and W_k is specific to horizon k. The matrix lets the model learn which relationships between context coordinates and future latent coordinates are useful for identifying a match.

The exponent turns the raw bilinear score into a positive compatibility value. As in equation 3.5, the objective normalizes these values across candidates to obtain a probability distribution. Higher compatibility means that a candidate receives a larger share of the probability assigned by the context.

Consider a batch containing B sequences. For a context from sequence i, the correct target is the future latent from sequence i. Future latents from the other sequences in the batch act as negative candidates. At horizon k, define the raw score

$$\left(z_{t+k}^{(j)}\right)^{\top} W_k c_t^{(i)},$$

where i identifies the context and j identifies a candidate future. The probability assigned to the correct candidate is

$$\frac{\exp(s_{ii}^{(k)})}{\sum_{j=1}^{B} \exp(s_{ij}^{(k)})}.$$

The loss for that context and horizon is the negative log-probability of the correct match. Averaging across contexts and trained horizons gives the multi-horizon InfoNCE objective:

$$-\frac{1}{|\mathcal{J}|} \sum_{(i,\, t,\, k) \in \mathcal{J}} \log \frac{\exp(s_{ii}^{(k)})}{\sum_{j=1}^{B} \exp(s_{ij}^{(k)})},$$

where $\mathcal{J}$ is the set of valid context, sequence, and horizon combinations in the batch. This is equation 3.5 aimed at a different task. Instead of identifying which view belongs to the same

observation, it identifies which future latent belongs to the current context.

The model is trained at several horizons. A one-step prediction head scores the latent at (t+1); a two-step head scores (t+2); a three-step head scores (t+3). Each head has its own matrix W_k. The context is shared, so its representation must support identification of future latents at each of these horizons.

The model does not directly regress the future latent with a squared-error objective. It does not take a predicted vector, feed that vector back into the model, and recursively generate a long sequence. It learns compatibility scores that rank the correct future above other candidates. Those are distinct tasks, and the evaluation must respect that distinction.

The implementation

In the repository, worldmodels/models/temporal_cpc.py defines TemporalCPC. Its forward method encodes a batch of image sequences, runs the GRU, and calculates cross-entropy over candidate future latents at each valid context and horizon.

listing 3.9 shows the context encoder and horizon-specific compatibility scoring used by CPC.

Listing 3.9 CPC context and compatibility scoring

```
z_seq = self.encode_sequence(x_seq)
c_seq, _ = self.gru(z_seq)

b, t, _ = z_seq.shape
cpc_sum = torch.zeros((), device=x_seq.device)
n_cpc = 0

for t_idx in range(0, t - self.k_steps):
    c_t = c_seq[:, t_idx]
    for k in range(self.k_steps):
        target = z_seq[:, t_idx + 1 + k]
        pred = c_t @ self.w_k[k].T
        logits = pred @ target.T
        labels = torch.arange(b, device=x_seq.device)
        cpc_sum = cpc_sum + F.cross_entropy(logits, labels)
        n_cpc += 1

return cpc_sum / n_cpc
```

The matrix multiplication produces a batch-by-batch score matrix. Row i contains the scores assigned by context i to every candidate future latent in that batch. The correct target lies on the diagonal, because context and future target come from the same sequence. Cross-entropy encourages the diagonal score to dominate its row.

The GRU is unidirectional. Although the implementation processes a tensor containing multiple time steps at once, the context output at index t is computed causally from the sequence up to that time step. Future latents are targets used to calculate the loss; they are not inputs to the context at t.

WARNING **Causality.** A context used to predict the future may contain the current

observation and earlier observations, but not the future target. If future frames influence the context, the model can identify the target using information that would not be available at prediction time.

What enters the experiment

The CPC experiment uses Cartpole Balance frames collected under a uniform random policy. The model receives sequences of images. It does not receive the actions that generated those sequences.

That omission matters. In a controlled physical system, the next state depends not only on the current state but also on the action applied and the system's dynamics. CPC here learns from a sequence generated by an unknown-to-the-model action stream. Its context must infer useful information from the visual history alone.

This is not a controlled comparison between action-conditioned and action-free prediction, so it does not measure how much actions would improve the results. Because the context receives no actions, the experiment evaluates future-latent identification under the sampled policy, not the consequences of alternative actions.

figure 3.15 traces the CPC learning signal from past observations through a context vector to candidate future latents.

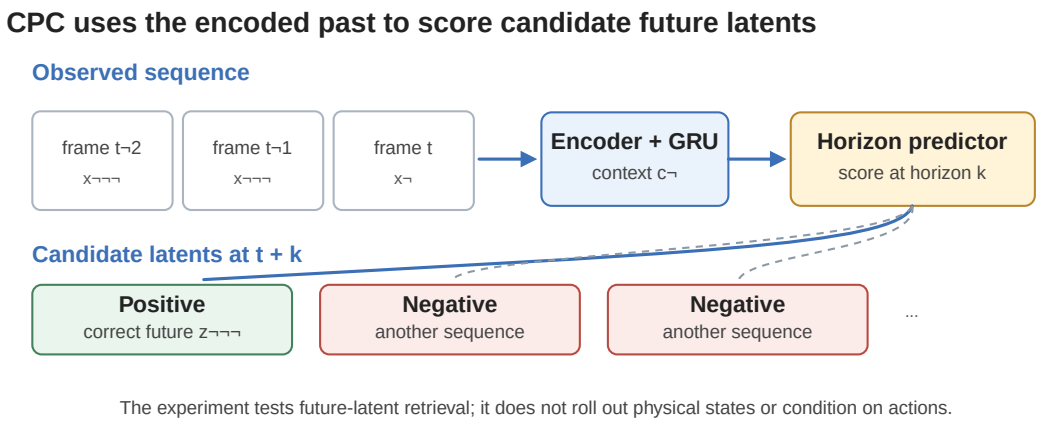


Figure 3.15 Contrastive predictive coding predicts a future latent

3.7.3. CPC evaluation: Future retrieval and state probes

A low CPC loss means that the correct future latent receives high probability under the model's candidate distribution. But the loss alone is harder to interpret than a direct retrieval measurement. We therefore evaluate the trained compatibility scores on held-out sequence windows and ask whether the correct future latent ranks above the other candidates.

This evaluation is aligned with the objective CPC actually trains. It is not a physical-state rollout

experiment.

Evaluation protocol

The committed run uses five trained model seeds: 0 through 4. Each model was trained for 20 epochs with a batch size of 32, learning rate 10^{-3}, encoder latent dimension 16, GRU context dimension 32, and three prediction horizons. The training configuration caps the data at 4,000 training sequence windows and 1,000 validation sequence windows, each of length 16 frames.

For the primary retrieval evaluation, the experiment selects 200 non-overlapping, episode-safe windows from the validation split. The selection is balanced across five validation episodes, with 40 windows per episode. The windows do not overlap within their source episodes, and they are shuffled into evaluation batches so that candidate sequences are not adjacent windows presented in temporal order. These 200 windows are not 200 independent trajectories; they come from only five validation episodes.

Evaluation compares each context's correct future latent with the corresponding future latents from other sequences in the same batch. There are six full batches of 32 candidates and a final batch of eight. The chance level is consequently computed for this query-weighted, in-batch protocol:

$$\frac{192(1/32) + 8(1/8)}{200} = 0.035 = 3.5\%$$

The random top-1 chance rate is therefore 3.5%, rather than $1/200$. The test does not rank each context against all 200 windows simultaneously. That distinction is part of the result, not a detail to omit from the caption.

Future-latent retrieval

The top-1 retrieval rate is the fraction of queries for which the matching future latent receives the highest score in its batch. Top-5 retrieval counts a match as correct if it appears among the five highest-scoring candidates. The positive-minus-hardest-negative margin measures the score gap between the correct future and the most competitive incorrect candidate. Because the logits are raw bilinear scores, that margin is a score difference, not a probability.

The consolidated evaluation below combines the state-probe comparison with horizon-wise retrieval and candidate-separation diagnostics. figure 3.16 brings these measurements together.

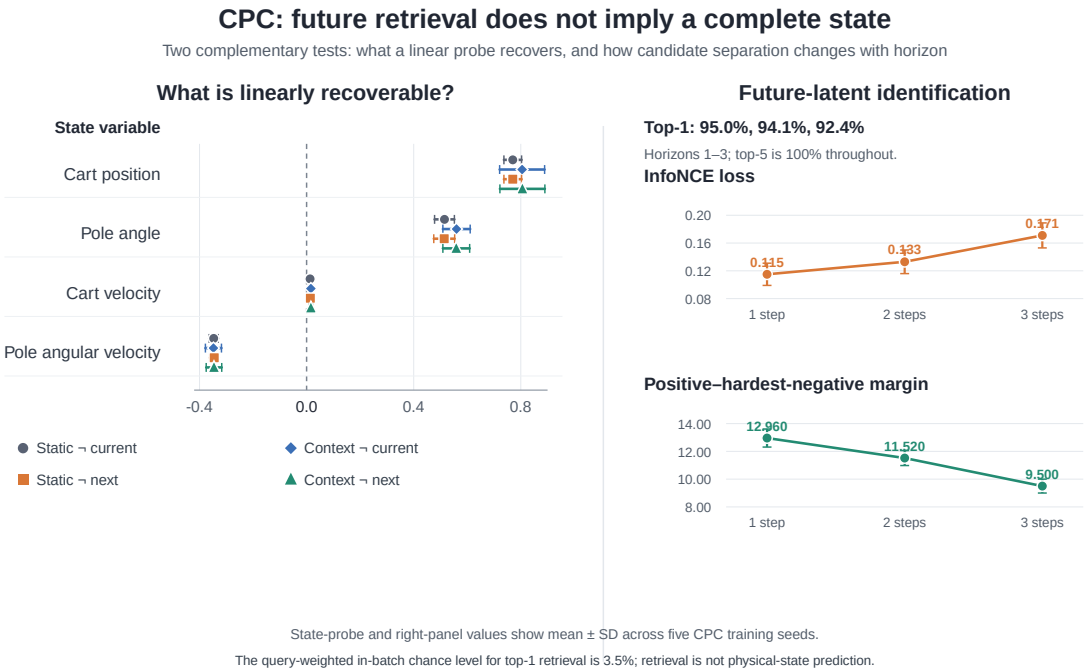


Figure 3.16 CPC state probes, retrieval, and candidate separation

The state-probe values and loss/margin curves summarize five trained seeds. The query-weighted in-batch chance level for top-1 retrieval is 3.5%. The 200 evaluation windows come from five validation episodes, so they are not 200 independent trajectory samples. Error bars show standard deviation across trained seeds, not a confidence interval over independent episodes.

Top-1 retrieval is 95.0%, 94.1%, and 92.4% at horizons one through three, respectively; top-5 retrieval is 100% at each horizon. Candidate separation weakens with horizon. These are rankings among in-batch future-latent candidates, not physical prediction scores.

The candidate protocol also limits the claim. Candidates come from only the selected validation windows and are compared within batches; the benchmark does not test retrieval across every possible future state or every possible trajectory. The five episodes contribute many windows, but windows from the same episode are not independent evidence of generalization to wholly new environments.

The recurrent context has a numerically higher current cart-position R^2 than the static encoder: 0.805 versus 0.770. Its mean next-state R^2 across the four state variables is also numerically higher, at 0.259 ± 0.016, compared with 0.238 ± 0.009 for the static representation. The per-variable measurements show that this change is limited. Cart velocity remains very weakly decodable, and pole angular velocity remains negative under both representations.

Negative R^2 means that the probe performs worse on the validation samples than a constant

predictor based on the validation-target mean. It does not prove that the representation contains no information about that variable; it says the fitted linear readout does not recover it effectively under this evaluation.

The variation across seeds also matters. The context's cart-position score varies more from seed to seed than the static score, and the study does not establish statistical significance for the differences between the two representations. It would be too strong to conclude that adding a GRU solves state estimation or makes velocity available. The measurement supports a more limited result: the recurrent context changes what is linearly available, but the change is not uniform across physical variables.

What the experiment establishes

The primary result is that the trained CPC compatibility function identifies matching future latents substantially above the protocol's 3.5% random-candidate chance level. Its retrieval performance remains high over three trained horizons, while the loss and margin indicate somewhat less separation at longer horizons.

The state probes provide a complementary picture. Current position is relatively well represented, but velocity remains poorly decoded from both the static latent and the recurrent context. High future-latent retrieval therefore does not imply that every physical variable is explicitly available in the representation. Nor does it establish that the model can predict a physical state recursively, remain stable across long rollouts, or support a useful planner.

The experiment also receives no action input. Because the frames were generated under a random policy, the learned context does not distinguish state-driven change from the effects of the actions taken. The study is not a controlled test of action-conditioning; it simply does not include it. We must not interpret the retrieval score as action-conditioned transition prediction or as a universal ceiling on what CPC can achieve.

WARNING

A recurrent state is not a memory guarantee. A GRU compresses a finite history into a fixed-width vector, and training on short windows does not establish that it will retain information over much longer intervals. This experiment measures future-latent retrieval on short sequences, not long-term memory.

CPC asks a context to identify a future embedding rather than enforce agreement between two views or reconstruct missing pixels. This makes the role of temporal prediction explicit, but retrieval alone does not show that a model can generate a physically accurate future or choose an action.

We now have several ways to train a visual representation. Contrastive learning encourages

agreement across views. Non-contrastive objectives add mechanisms to prevent collapse. Masked prediction trains a model to reconstruct hidden pixels, and CPC asks a context to identify a future representation.

These methods differ in what they reward, but the agent should not need a different interface for every training objective. Once an encoder has been trained, the agent needs a component that accepts an observation and returns a representation. If the agent maintains a recurrent state, the same component must also update that state as new observations arrive.

We can expose a learned encoder through a small interface: map an observation to a latent vector and, optionally, update a recurrent hidden state. This supplies a representation; it does not implement learned transition dynamics or planning.

3.8. Assembling the encoder

Once an encoder has been trained, downstream code needs a stable way to obtain its representation without depending on the objective that produced it.

3.8.1. A common encoder interface

The shared interface is implemented in the encoder module. The Encoder class wraps a backbone, such as the ConvEncoder used in contrastive training. It exposes two methods:

`encode(obs)` maps an observation to its latent representation.

`update(state, obs)` processes the observation and returns the updated agent state.

The second method can operate in either of two ways. Without a recurrent layer, it simply returns the encoded observation. With a recurrent layer, it passes the representation and previous recurrent state through a GRU cell. On the first call, when no previous state exists, it initializes the hidden state to zeros.

listing 3.10 shows the interface calls for an encoder with or without recurrent state.

Listing 3.10 Two ways to use the encoder

```
z = encoder.encode(observation)
state = encoder.update(previous_state, observation)
```

The first call returns a visual representation. The second does the same and, when a recurrent wrapper is configured, also updates its hidden state. Without that wrapper, there is no memory across calls. The hidden state is learned internal state, not the simulator’s ground-truth physics state.

The optional GRU carries a learned hidden state across observations; without it, calls are stateless.

This hidden state is not the simulator's ground-truth qpos/qvel. The interface is independent of the training objective, but it supplies neither action-conditioned transition dynamics nor planning.

3.8.2. Verifying the interface

A smoke test loads a saved contrastive encoder, wraps it with a 32-dimensional GRU, and processes ten validation frames sequentially, carrying the hidden state from one frame to the next.

The resulting recurrent-state shape is `(1, 32)`.

This is an interface check, not an evaluation of representation quality. It verifies that the checkpoint loads, observations are encoded, and recurrent state carries across the tested sequence. It does not establish that the representation is sufficient for control, that recurrent state captures all relevant history, or that the system can act successfully. Representation quality must still be judged by held-out probes and by the predictive task it is intended to support.

3.8.3. What each objective encourages the encoder to preserve

The experiments show why a common interface does not imply that all encoders learn equivalent representations. The training objective determines which relationships are rewarded. table 3.3 summarizes the resulting trade-offs.

Table 3.3 Representation objectives and their trade-offs

Objective	**Information encouraged**	**May discard**	**Potential failure mode**
Pretext tasks	Task-defined structure	Variables irrelevant to the pretext	Shortcut features
Contrastive (NT-Xent)	Structures preserved across views	Information removed by augmentation	False negatives and aggressive invariance
Non-contrastive methods	Invariant, non-collapsed coordinates	Redundant coordinates	Collapse if the anti-collapse mechanism fails
Masked autoencoding	Information useful for reconstruction	Details irrelevant to pixel recovery	Capacity spent on predictable pixels
Temporal CPC	Information useful for future prediction	Unpredictable innovations	Action-unconditioned prediction ceiling

This table is not a ranking. Each row summarizes the direct pressure imposed by an objective

family and a possible failure mode; whether a model exhibits that failure must be measured.

Pretext tasks define their own prediction target. That target can create useful supervision from the structure of observations, but it can also reward shortcuts that solve the pretext without preserving the information needed downstream.

Contrastive learning rewards agreement across the views constructed by the augmentation pipeline. This is useful when the chosen transformations preserve task-relevant distinctions. It becomes a problem when an augmentation removes information the agent needs, or when different frames that look similar become false negatives.

Non-contrastive methods demonstrate that negatives are not the only way to prevent collapse. SimSiam, BYOL, Barlow Twins, and VICReg use different mechanisms to constrain the learned representation. Those mechanisms introduce their own design choices, and the reduced-scale experiments in this chapter do not establish a universal ranking of the methods.

Masked autoencoding rewards reconstruction of missing pixel content. It may devote capacity to background or other predictable image details that do not make physical variables easy to decode. The Cartpole experiment found that reconstruction quality and state-probe quality are distinct measurements.

CPC changes the target from reconstructing an observation to identifying its future representation from a context. The retrieval experiment shows that the correct future latent can be identified at high rates within the tested candidate batches. Its state probes remain weak for velocity, and the model receives no action input. Future-latent identification alone is therefore not a guarantee of a complete predictive state.

The common pattern is that an objective cannot be judged solely by whether its loss decreases. We need to examine what its representation preserves, what it discards, and whether those properties match the agent's requirements.

3.8.4. What the experiments establish

The measurements provide partial evidence about the learned representations, but they do not settle every question. The following summary separates what was measured from what remains unverified.

The measurements address parts of the state contract, but leave several stronger claims untested. table 3.4 maps each requirement to evidence and its limits.

Table 3.4 Chapter 2's state contract: evidence and remaining questions

Contract question	Evidence in this chapter	What remains untested
Sufficiency	Frozen-feature linear probes measure whether simulator variables can be decoded from the representation.	Low R^2 does not prove the information is absent; no experiment demonstrates successful control or planning.
Compactness	The latent-dimension sweep measures how probe performance changes with representation size.	It does not establish a globally minimal state or directly measure mutual information.
Aliasing and memory	Single-frame and stacked-frame probes test selected ambiguities; CPC adds a recurrent context and a future-latent retrieval task.	Stacking two frames does not reliably recover velocity in this configuration. No formal Markov-sufficiency test is performed.
Temporal prediction	CPC retrieves the matching future latent at 95.0%, 94.1%, and 92.4% top-1 across the three tested horizons.	Retrieval is not a recursive physical-state rollout, a long-horizon consistency test, or proof of state sufficiency.
Action-conditioned dynamics	The chapter distinguishes the encoder interface from the larger transition-and-planning system.	CPC receives observations, not actions. Action-conditioned prediction and planning utility are not measured.

figure 3.17 places the encoder in a larger control loop and distinguishes representation from action-conditioned dynamics and planning.

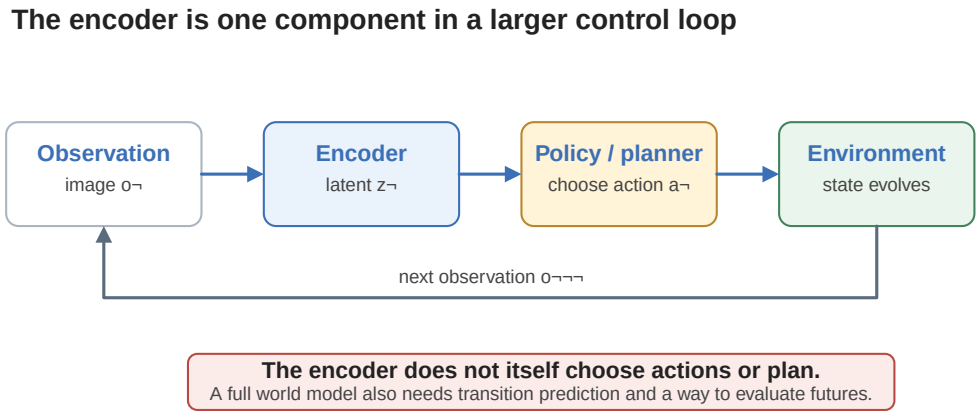


Figure 3.17 Using a learned encoder in a control loop

Exercises

These exercises connect the chapter's mathematical objectives to the behavior of the representations they train. For each experiment, write down a hypothesis before running the code, preserve the configuration and random seeds, and distinguish a measured result from an explanation of why it occurred.

1. Deriving the contrastive gradient

Starting from equation 3.5, derive the gradient of the NT-Xent loss with respect to the anchor representation z_1.

Separate the gradient into the contribution from the positive example and the contributions from the negative examples. Identify which terms pull the positive closer and which push negative candidates away. Explain how the temperature τ changes the relative weights assigned to candidates.

2. Temperature and negative-example weighting

Consider InfoNCE as the temperature approaches infinity, $\tau \to \infty$. Show what happens to the softmax probabilities when the similarities are divided by an increasingly large temperature.

Predict the effect on the gradients and on the learned representation. Then increase the temperature in the contrastive training experiment and compare the loss curve and frozen-feature state probes against a baseline. Keep the data, architecture, and training budget fixed.

3. Removing a VICReg term

VICReg combines invariance, variance, and covariance penalties. Which term prevents the trivial constant representation from satisfying the objective at zero cost? What failure would you expect if the covariance penalty were removed while the other terms remained?

State your predictions for latent spread, effective rank, and state-probe performance. Then modify the experiment in ch03/08_vicreg.py, run it with the same seeds and budget, and compare the measurements with the complete objective. Distinguish a reduction in redundancy from a genuine collapse.

4. When augmentation removes state information

The contrastive view pipeline uses crop shifts of up to three pixels. Suppose the maximum shift increases to 16 pixels, while Cartpole remains rendered at 64×64 resolution.

Predict how this change will affect the cart-position probe and explain your prediction using the chapter's central principle: the objective rewards information preserved by the view pipeline. Then run the experiment with the larger shift, keeping all other settings fixed. Report the cart-position

score across seeds and compare it with the baseline.

5. Choosing the latent dimension

The bottleneck sweep measures state-probe performance at latent dimensions $d \in 2, 4, 8, 16, 64, 256$ for Cartpole Balance and Cheetah Run.

At what dimension does Cartpole's probe approach a plateau? Does Cheetah show the same pattern within the tested range? Relate the results to the tasks' four- and eighteen-dimensional physical states, respectively, and explain why the optimal latent dimension need not equal the number of state variables.

Use figure 3.14 and ch03/results/bottleneck_sweep_metrics.json. Identify the smallest tested dimension that appears to approach saturation for each task, and state what further experiment would be needed if performance is still rising at the largest dimension.

6. Extending CPC's prediction horizon

Modify the CPC experiment to predict five future steps instead of three. Predict how future-latent retrieval, InfoNCE loss, and the positive-versus-hardest-negative score margin will change as the horizon increases.

The reported evaluation uses three prediction horizons. The full evaluation protocol and per-window records are retained with the experiment results.

Explain the difference between identifying the correct future latent among candidates and predicting a physical state accurately. In particular, state why a strong retrieval result does not establish accurate recursive prediction, action-conditioned dynamics, or planning utility.

7. Testing temporal aliasing on Finger Spin

The existing aliasing diagnostic compares single-frame features with features formed from two consecutive Cartpole frames. Its current runner hardcodes the Cartpole datasets and four Cartpole state-variable names. Extend it to support Finger Spin before running the experiment, including the correct variable mapping and the periodicity-aware evaluation for the spinner's angle.

Predict which Finger Spin state variables will be easier to decode from a single frame and which, if any, will benefit from stacking two frames. Then compare single-frame and stacked-frame probe scores across the same three seeds.

NOTE

Interpreting Finger Spin probes. Finger Spin's rendered spinner has π-periodic symmetry: a static image does not uniquely identify its absolute 2π angle. Account for this when interpreting the probe scores. Distinguish a variable that is unobservable from a single image because of the rendering symmetry from one

that is merely difficult to decode under the tested model and training budget.

appendix A: InfoNCE bound and gradient details

This appendix gives the full derivation of the InfoNCE bound and the associated gradient. It uses the notation in table 3.1. The bound depends on its sampling assumptions; the experiment's trajectory batch is not an ideal independent-negative sampler.

Where the bound comes from

Sidebar: A proof of the InfoNCE bound

Suppose one of $K+1$ candidates is drawn from the conditional distribution $p(b \mid a)$, while the others are drawn independently from the marginal distribution $p(b)$. Let $r_j = p(b_j \mid a) / p(b_j)$, and let $S = \sum_{j=0}^{K} r_j$.

Step 1: The best possible classifier. Given the anchor and candidates, Bayes' rule assigns candidate j probability r_j / S of being the positive. Any classifier q_j incurs cross-entropy at least as large as this true posterior:

$$\mathcal{L}(f) \geq -\mathbb{E} \log \frac{r_m}{S}.$$

Step 2: Express that cross-entropy using mutual information. The expected log likelihood ratio of a positive pair is $I(a; b)$, so the expression on the right is

$$\mathbb{E} \log S - I(a; b).$$

Step 3: Bound the remaining term. Under the reference distribution where all $K+1$ candidates are independently sampled from the marginal $p(b)$, write $T = S / (K+1)$. The actual candidate-set distribution has density T relative to this reference distribution, so

$$\mathbb{E}[\log S] = \log(K+1) + \mathbb{E}_0[T \log T] \geq \log(K+1),$$

because $\mathbb{E}_0[T=1]$ and $t \log t$ is convex, so Jensen's inequality makes $\mathbb{E}_0[T \log T]$ nonnegative. Combining the three steps yields

$$\mathcal{L}_{\text{InfoNCE}} \geq \log(K+1) - I(a; b),$$

which rearranges to equation 3.6.

The gradient can also be made precise. The result in equation 3.7 separates attraction toward the positive from repulsion by negatives. For an anchor i, define the score $s_{ij} = (z_i, z_j)$. Differentiating the cross-entropy in equation 3.5 with respect to a candidate score gives

$$\frac{p_i(j) - \mathbf{1}[j = i^+]}{\tau},$$

where $\mathbf{1}[\cdot]$ equals one if its condition is true and zero otherwise. Applying the chain rule yields the gradient with respect to the anchor representation:

For cosine similarity,

$$\frac{1}{\| u \|_2}\left(\frac{v}{\| v \|_2} - (u, v)\frac{u}{\| u \|_2}\right).$$

The gradient is perpendicular to u, so each score's local update changes the direction of the vector rather than its length to first order.

Why this bound is not a physical-state guarantee

The lower bound concerns mutual information shared by paired-view representations under the stated sampling assumptions. It does not establish that cart position, angle, velocity, or an action-conditioned predictive state is linearly available. The data-processing inequality also limits shared information to information that survives the view-generation pipeline. These are the precise reasons the chapter follows the training objective with held-out state probes.

References and figure credits

The sources below document the benchmark, learning objectives, implementation, and figure provenance used in this chapter.

Benchmark and environment

The benchmark and simulator documentation are:

- Tunyasuvunakool, S. et al. (2020). "dm_control: Software and tasks for continuous control." *Software Impacts*, 6, 100022. https://doi.org/10.1016/j.simpa.2020.100022
- Google DeepMind, `dm_control` source repository: https://github.com/google-deepmind/dm_control
- MuJoCo documentation: https://mujoco.org/

Self-supervised objectives

The learning objectives discussed in the chapter are described in these papers:

- Gui, J. et al. (2024). "A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12), 9052–9071. https://doi.org/10.1109/TPAMI.2024.3415112. Preprint: https://arxiv.org/abs/

2301.05712

- LeCun, Y. (2020). “Self-Supervised Learning.” ICLR 2020 keynote. Official session page: https://iclr.cc/virtual_2020/speaker_7.html. Public video recording: https://www.youtube.com/watch?v=8TTK-Dd0H9U.
- Chen, T. et al. (2020). “A Simple Framework for Contrastive Learning of Visual Representations.” *ICML*. https://arxiv.org/abs/2002.05709
- van den Oord, A., Li, Y., and Vinyals, O. (2018). “Representation Learning with Contrastive Predictive Coding.” https://arxiv.org/abs/1807.03748
- Chen, X. and He, K. (2021). “Exploring Simple Siamese Representation Learning.” *CVPR*. https://arxiv.org/abs/2011.10566
- Grill, J.-B. et al. (2020). “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning.” *NeurIPS*. https://arxiv.org/abs/2006.07733
- Zbontar, J. et al. (2021). “Barlow Twins: Self-Supervised Learning via Redundancy Reduction.” *ICML*. https://arxiv.org/abs/2103.03230
- Bardes, A., Ponce, J., and LeCun, Y. (2022). “VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning.” *ICLR*. https://arxiv.org/abs/2105.04906
- He, K. et al. (2022). “Masked Autoencoders Are Scalable Vision Learners.” *CVPR*. https://arxiv.org/abs/2111.06377

Implementation and reproducibility

The implementation and numerical provenance are available in these companion files:

- Canonical implementation modules: `worldmodels/` in the code companion.
- Experiment runners, outputs, and numerical provenance: `ch03/` and `ch03/results/`.
- Current figure assets: `ch03/figures/`.
- Exact runtime and rendering guidance: `ch03/README.md`. Hardware-dependent runtime estimates must be reported as configuration- and machine-specific, not guaranteed speedups.

Figure credits

The original diagrams explain the objectives and data flow; experimental plots use the committed results. Any adapted figure must carry attribution in its caption.

Updated taxonomy sources

The updated taxonomy discussion also draws on the following sources:

- Uelwer, T., Robine, J., Wagner, S. S., Höftmann, M., Upschulte, E., Konietzny, S., Behrendt, M., et al. (2025). “A survey on self-supervised methods for visual representation learning.” **Machine Learning**, 114, Article 111. https://doi.org/10.1007/s10994-024-06708-7
- Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., et al. (2025). “V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning.” Meta AI research publication, June 11, 2025. https://ai.meta.com/research/publications/v-jepa-2-self-supervised-video-models-enable-understanding-prediction-and-planning/

Summary

- Pretext and contrastive learning: Data-created targets and matching-view classification impose useful but limited constraints. In the three-seed Cartpole run, NT-Xent loss fell from about 3.78 to 3.11, but positions were more linearly accessible than velocities and stacking two frames did not reliably improve velocity decoding.
- Collapse and its remedies: Attraction alone admits a constant-representation solution. SimSiam, BYOL, Barlow Twins, and VICReg use different mechanisms to limit collapse or redundancy; assess them with feature spread, effective rank, and held-out probes, not training loss alone.
- Masked reconstruction: The MAE hides 75% of image patches and predicts their pixels. Its standalone Cartpole probe reached mean R^2 of 0.170 ± 0.004, showing that reconstruction quality and physical-state decodability are different measurements.
- Capacity and temporal prediction: Cartpole probe R^2 rose from about -0.008 at $d = 2$ to 0.321 at $d = 64$ and was similar at $d = 256$; Cheetah still reached 0.422 at $d = 256$. CPC retrieval was 95.0%, 94.1%, and 92.4% at horizons one, two, and three versus 3.5% chance, but this is not evidence of complete state recovery, action-conditioned dynamics, or planning utility.
- Encoder interface: The `encode(obs)` and `update(state, obs)` methods expose a visual representation and optional recurrent state. They do not supply action-conditioned dynamics or planning.